



Programme

Master of Software Engineering

Course

**MSE800 Professional Software Engineering
(Level 8, 30 Credits)**

Assessment 2

Object-Oriented Project

(Classification - Group)

Student: Wijemuni Rukshan Madushanka De Silva

Student ID: 270802602

Te Tiriti o Waitangi Commitment

This project upholds the three principles of Te Tiriti o Waitangi: Partnership (Kawanatanga) through equal collaboration between team members; Participation (Tino Rangatiratanga) through Te Reo Māori bilingual interface throughout the platform; and Protection (Ōritetanga) through the Cultural Guardian module that safeguards Wāhi Tapu (sacred Māori sites) from unreviewed maintenance dispatch

Abstract

The FixItPublic system which New Zealand local councils use to handle civic issues developed through the MSE800 Professional Software Engineering group project. The platform enables citizens to report public infrastructure problems which include potholes and broken streetlights and graffiti and flooding through a responsive web application that uses GPS location detection and photo upload capabilities. The system uses a GPT-4o artificial intelligence engine to analyze each submission in a non-blocking background thread which automatically routes reports to the correct maintenance teams after it verifies crew assignment and escalation risk assessment and confidence score generation of the plain language summary.

The platform supports English and Te Reo Māori as its two language options because of the Te Tiriti o Waitangi requirement to enable public involvement through partnership, and the system includes a Cultural Guardian module that uses a ray-casting point-in-polygon algorithm to compare GPS coordinates with four Auckland Wāhi Tapu (sacred Māori sites) polygons which block maintenance teams from working on areas until they receive proper assessment. The system operates through Azure Kubernetes Service which utilizes a GitHub Actions pipeline for its automated process of continuous integration and deployment.

My work as Frontend Lead and QA Engineer delivered complete React 18 user interface development together with two authentication context systems and Google Maps integration and three-step report submission form and citizen dashboard with avatar upload function and complete QA testing which achieved 100% success through 28 test cases. I built three Python decorators through my work when I developed `@log_request` and `@require_council_role` and `@cache_response` decorators which proved my proficiency in advanced Python meta-programming for MSE800 decorator assessment requirements. The report presents my individual contributions together with a comprehensive evaluation of my partner Nirmal Unagalle's backend and DevOps work which demonstrates our complete understanding of the system.

Table of Contents

Abstract.....	2
1. Introduction	6
1.1 Background	6
1.2 Objectives	6
1.3 Scope.....	6
1.4 Stakeholders	7
1.5 Assumptions and Constraints	7
2. Requirements Analysis	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	8
2.3 MoSCoW Prioritisation	9
2.4 Technology Stack.....	9
2.5 Original Plan vs Delivered System	11
3. System Design and Architecture.....	12
3.1 Architectural Style.....	12
3.2 Design Patterns	13
3.2.1 Decorator Pattern (My Contribution)	13
3.2.2 Context Pattern (React).....	13
3.2.3 Observer Pattern (Django Signals).....	14
3.3 UML Class Diagram.....	14
3.4 Use Case Diagram	15
3.5 Sequence Diagram — Ticket Submission.....	15
3.6 Entity Relationship Diagram.....	16
4. Implementation	17
4.1 My Frontend Contributions	17
4.1.1 Authentication System	17
4.1.2 Report Submission (ReportIssuePage.jsx).....	17
4.1.3 Citizen Dashboard (CitizenDashboardPage.jsx)	18
4.1.4 Maps (PublicMap.jsx and AdminMap.jsx)	18
4.2 My Python Decorators (backend/api/decorators.py)	18
4.3 Nirmal's Backend Contributions	20

4.3.1 GPT-4o AI Integration (signals.py).....	20
4.3.2 Cultural Guardian (cultural_guardian.py)	20
4.3.3 Google OAuth (google_auth.py)	20
4.4 API Endpoints	21
5. Deployment and Release	28
5.1 Containerization (Docker)	28
5.2 Azure Cloud Infrastructure.....	28
5.3 CI/CD Pipeline (GitHub Actions).....	29
6. Agile Development Process.....	30
6.1 Agile Methodology and Framework.....	30
6.2 Sprint Overview and Velocity	31
6.3 Sprint-by-Sprint: Planned vs Delivered	32
6.4 Version History	35
7. Testing	35
7.1 Testing Strategy	35
7.2 Test Cases and Results	36
7.3 Bug Fixes Identified During Testing	38
8. Technical Challenges and Innovative Solutions	39
8.1 GPT-4o AI Integration	39
8.2 Cultural Guardian Module.....	39
8.3 Python Decorator Architecture	39
9. Software Evolution and Maintenance	40
9.1 Maintenance Activities	40
9.2 Version Control	40
9.3 Future Roadmap	41
10. Limitations and Future Work.....	41
10.1 Known Limitations	41
10.2 Future Enhancements.....	42
11. Te Tiriti o Waitangi.....	42
11.1 Partnership — Kāwanatanga	42

11.2 Participation — Tino Rangatiratanga (My Contribution)	43
11.3 Protection — Ōritetanga	43
12. Reflection	43
12.1 Technical Growth	43
12.2 Cultural Learnings	44
12.3 Teamwork	44
13. Conclusion.....	44
14. References.....	45
15. Appendix.....	47
15.1 GitHub Repository	47
15.2 Live Deployment	47
15.3 Key Source Files	47
15.4 Git Contribution Log (Selected Key Commits)	47
15.5 Monday.com Sprint Board	48
15.6 QA Report	49

1. Introduction

1.1 Background

New Zealand local councils handle more than 1000 requests for public infrastructure maintenance every year. The existing reporting systems which include telephone hotlines and paper-based systems cause operational delays through their failure to properly log and route and monitor issues. The solution for this problem exists in FixItPublic which develops a digital civic engagement platform that allows citizens to report problems through their mobile devices by using GPS tracking and photographic evidence and live update features.

Students in the MSE800 Professional Software Engineering course must complete their project work in pairs. Rukshan De Silva served as Frontend Lead and QA Engineer; Nirmal Unagalle served as Backend Lead and DevOps Engineer. The team completed their platform development by creating testing procedures and deploying their finished product to Microsoft Azure which they accomplished through eight weeks of agile development work.

1.2 Objectives

The system will provide users with an easy-to-use reporting system which enables them to report issues through multiple steps while using GPS tracking and photo uploading and selecting from different categories of problems. The system uses artificial intelligence to automatically direct reports to the correct maintenance teams for handling. The dashboard provides council administrators with complete access to map data and AI tools which support their decision-making process. The system will provide bilingual support for English and Te Reo Māori according to the requirements of Te Tiriti o Waitangi. The organization will create a secure and scalable cloud-based platform which operates on Azure Kubernetes Service. The students must show their advanced Python decorator skills to fulfill the curriculum requirements of MSE800.

1.3 Scope

The platform supports three user roles which include citizens for public issue reporting and council administrators for ticket management and crew assignment and superusers who handle complete system administration. The system uses Google Maps for location services and OpenAI GPT-4o for AI analysis and Azure Blob Storage for photo management and Namecheap SMTP for email notifications. The system provides a Progressive Web App manifest together with a service worker to enable future mobile access without internet connection.

1.4 Stakeholders

Table 1. Stakeholders Table

Stakeholder	Role	Needs
Citizens	Primary users	Easy reporting, status tracking, email updates
Council Administrators	System operators	Efficient ticket management, AI recommendations, map overview
Maintenance Crews	Field workers	Clear, accurate crew assignments from the system
Māori Community	Cultural stakeholders	Wāhi Tapu protection, Te Reo Māori interface
System Administrators	Technical owners	User management, system monitoring, AI log access

1.5 Assumptions and Constraints

- The system operates exclusively through web browsers without any native mobile application development.
- The system depends on Google Maps API and OpenAI GPT-4o as external components which will function at a reduced level if these components become unavailable.
- Active subscription and verified credentials are necessary to access Azure cloud services.
- The system will execute AI analysis without interruptions when the OPENAI_API_KEY exists as an optional component.
- The Cultural Guardian site polygons function as sample representations of Auckland region boundaries while actual production work will obtain data from council GeoJSON API.

2. Requirements Analysis

2.1 Functional Requirements

Table 2. Functional Requirements

ID	Requirement	Priority	My Work
FR1	Citizens can register with email, phone, password (strength validated)	MUST	RegisterPage.jsx with validation
FR2	Citizens and admins can log in with JWT authentication	MUST	LoginPage.jsx, CitizenAuthContext

FR3	Citizens can submit multi-step issue reports with GPS and photos	MUST	ReportIssuePage.jsx (3-step form)
FR4	System routes tickets to maintenance crews automatically	MUST	CATEGORY_CREW_MAP + GPT-4o (Nirmal)
FR5	Admins can view, filter, assign, and update ticket status	MUST	AdminDashboard (Nirmal/Rukshan)
FR6	System sends email notifications for key events	SHOULD	emails.py (Nirmal)
FR7	Public map displays all geolocated tickets with filters	SHOULD	PublicMap.jsx
FR8	GPT-4o analyses each ticket in background thread	SHOULD	signals.py (Nirmal)
FR9	Cultural Guardian flags tickets near Wāhi Tapu sites	COULD	cultural_guardian.py (Nirmal) + admin UI (Rukshan)
FR10	Citizens can manage profile, avatar, and password	SHOULD	CitizenDashboardPage.jsx

2.2 Non-Functional Requirements

Table 3. Non-Functional Requirements

ID	Requirement	Implementation
NFR1	Security: JWT with 60-min access + 7-day rotating refresh tokens	SimpleJWT with blacklist
NFR2	Rate limiting: prevent abuse of public endpoints	AnonRateThrottle (5/hr register, 10/hr tickets)
NFR3	Performance: public stats endpoint must respond quickly under load	@cache_response(300) decorator
NFR4	Scalability: containerised for horizontal scaling	Docker + AKS deployment
NFR5	Cultural compliance: bilingual UI and Wāhi Tapu protection	Te Reo Māori labels + Cultural Guardian
NFR6	Auditability: all key API requests logged with user and timing	@log_request decorator
NFR7	Reliability: AI failure must not break ticket submission	Graceful degradation in signals.py

NFR8	Data privacy: no raw PII sent to OpenAI	redact_pii() called before API request
------	---	--

2.3 MoSCoW Prioritisation

Table 2.3 shows the full MoSCoW analysis for the project requirements.

Table 4. MoSCoW analysis for the project requirements

MUST	Citizen registration and login with JWT	Implemented
MUST	Multi-step issue report form with GPS and photos	Implemented
MUST	Admin ticket management (view, assign, update)	Implemented
MUST	Bilingual UI (English / Te Reo Māori)	Implemented
SHOULD	GPT-4o AI ticket analysis (background thread)	Implemented
SHOULD	Google Maps integration (public + admin)	Implemented
SHOULD	Email notifications for ticket events	Implemented
SHOULD	Citizen profile management and avatar upload	Implemented
COULD	Cultural Guardian Wāhi Tapu cross-reference	Implemented (stub polygons)
COULD	Spatial deduplication (50m radius clustering)	Planned
WON'T	Native mobile application	Out of scope

2.4 Technology Stack

Table 2.4 maps every technology used in FixItPublic against its plan status (Planned / Extended / Extra) and its specific role in the system.

Table 5. Planned / Extended / Extra technology used in FixItPublic against its plan status

Layer	Technology	Plan Status	Role in FixItPublic
Backend	Python 3.11 + Django 4.2	Planned	Core backend; ORM, migrations, admin, signal system.
Backend	Django REST Framework 3.14	Planned	RESTful API: serializers, viewsets, throttling, permissions.
Backend	djangorestframework-simplejwt	Extra	JWT access/refresh tokens; blacklisting on logout.
Backend	OpenAI GPT-4o	Extended (LLM)	Structured-output AI engine; crew routing, PII redaction, confidence score.
Backend	google-auth + django-storages[azure]	Extra	Google OAuth 2.0 verification; Azure Blob for photos and avatars.

Frontend	React 18 + Vite + Tailwind CSS v3	Planned (React)	Fast HMR builds; utility-first CSS; responsive bilingual UI.
Frontend	React Router v6 + Axios	Extra	PrivateRoute guard; silent JWT refresh interceptor on 401.
Frontend	Google Maps JS API + Nominatim	Planned	Interactive map pins; free reverse-geocoding (OpenStreetMap).
Database	PostgreSQL 15 — Azure Flexible Server	Planned	SSL-enforced; 9 ORM migrations across 4 sprints.
Infra	Docker + Docker Compose	Planned	Separate backend and frontend containers; Compose for local dev.
Infra	Azure Kubernetes Service (AKS)	Extended (optional)	Two-pod microservices; Recreate / RollingUpdate deploy strategies.
Infra	Azure Container Registry + Blob	Extra	SHA-tagged image registry; object storage for media files.
CI/CD	GitHub Actions	Extra (not planned)	Automated build-push-deploy on every push to main.
Project Mgmt	Monday.com + Git/GitHub	Planned	Backlog, story points, burndown; PR-based branch workflow.
Testing	BDD Given/When/Then (28 cases)	Extended	Structured manual end-to-end QA on live Azure environment.
External	SMTP Email + Google OAuth 2.0	Planned / Extra	Status-change emails (opt-in toggle); one-click citizen sign-in.

The project shifted from its initial plan of using Azure Container Apps to AKS after Nirmal's pre-project proof-of-concept cluster showed that the latter would work. The project needed Kubernetes to be used as a core system element instead of its prior function as a secondary component. The AI system expanded its capabilities from basic LLM text support to full GPT-4o structured-output engine with PII redaction and daemon threads and confidence scoring and a reasoning audit trail.

2.5 Original Plan vs Delivered System

Direct feature-level comparison between the original proposal and final delivery. 'Exceeded' = delivered more than scoped; 'Extra' = not in the original plan.

Table 6. Original Plan vs Delivered System

Feature	Originally Planned	Final Status	Details
Citizen issue reporting form	Planned	Delivered	3-step form with GPS, category, up to 5 photos, AI animation feedback
Admin dashboard	Planned	Delivered	Full ticket management: view, filter, sort, assign crew, update status, escalation
Google Maps / OpenStreetMap integration	Planned	Delivered	Both: Google Maps for display, Nominatim for reverse-geocoding (free)
Email notifications	Planned	Delivered	SMTP notifications with per-user opt-in/out toggle stored in database
PostgreSQL database	Planned	Delivered	Azure PostgreSQL Flexible Server, SSL, 9 migrations, full relational schema
Docker containerisation	Planned	Delivered	Two optimised containers (Django + React/Nginx), Docker Compose for local dev
Azure hosting	Planned	Exceeded	AKS (full Kubernetes) instead of Container Apps; two-pod microservices deployment
Kubernetes	Optional	Delivered	Fully implemented, not skipped; Recreate strategy for zero migration-conflict deploys
LLM AI integration	Limited	Exceeded	GPT-4o full integration: structured JSON output, PII redaction, confidence score, reasoning audit trail, background daemon thread (non-blocking)
CI/CD pipeline	Not planned	Extra	GitHub Actions: automated build → ACR → AKS deploy on every push to main
Google OAuth sign-in	Not planned	Extra	One-click Google sign-in for citizens; admin accounts excluded from auto-creation
Avatar upload	Not planned	Extra	Citizen avatar upload to Azure Blob Storage with cache-control headers and instant preview
Bilingual Te Reo Maori UI	Not planned	Extra	All 8 issue categories include Te Reo Maori subtitles throughout the interface

Cultural Guardian module	Not planned	Extra	Ray-casting point-in-polygon checks GPS coordinates against 4 Auckland Wahi Tapu polygons; flags tickets for manual review
Python decorator architecture	Not planned	Extra	@log_request, @require_council_role, @cache_response — MSE800 decorator assessment criterion
Rate limiting / throttling	Not planned	Extra	AnonRateThrottle: 5/hr register, 10/hr ticket submission; prevents abuse
Token refresh interceptor	Not planned	Extra	Axios interceptor silently refreshes expired JWT tokens and replays failed request
Monday.com sprint tracking	Planned	Delivered	Used for all 4 sprints: backlog, story points, burndown
Custom DNS + HTTPS (Cloudflare)	Planned	In progress	Noted in project update email; infrastructure ready, DNS configuration pending

3. System Design and Architecture

3.1 Architectural Style

FixItPublic uses a five-layer architectural framework. The Presentation Layer uses React 18 to create a single-page application which Vite builds and Tailwind CSS styles. The API Gateway Layer uses Django REST Framework to provide JWT authentication and rate throttling and CORS handling and custom cross-cutting decorators. The Business Logic Layer processes tickets through GPT-4o AI analysis which operates in background daemon threads and handles email notifications and protects cultural content. The Data Access Layer controls PostgreSQL and Azure Blob Storage through Django's ORM and cache framework. The Infrastructure Layer operates Azure Kubernetes Service which uses GitHub Actions for CI/CD automation.

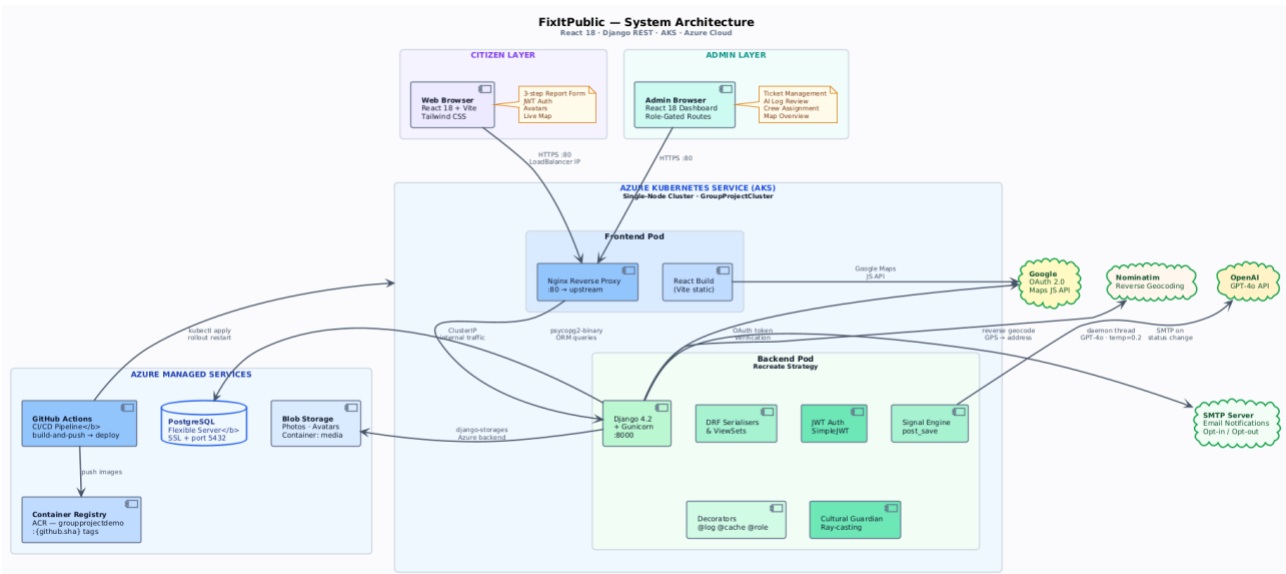


Figure 1. FixItPublic System Architecture (Layered)

3.2 Design Patterns

3.2.1 Decorator Pattern (My Contribution)

The three custom Python function decorators create three cross-cutting concerns because they enable activity logging and user access control and cache data storage without modifying the core view functions. The Open/Closed Principle allows view extensions through decorators while preventing any modifications to existing code. The decorators establish a systematic stacking method which allows multiple concerns to be applied to a single view function through declarative techniques.

3.2.2 Context Pattern (React)

The two React Context providers `CitizenAuthContext` and `AdminAuthContext` handle authentication state management as separate systems which maintain separation between citizen and admin portal access. The two contexts each maintain their own authentication tokens in separate `localStorage` storage locations while they offer their portal-specific login/logout and `isAuthenticated` authentication state functions.

3.2.3 Observer Pattern (Django Signals)

Django's `post_save` signal on `MaintenanceTicket` triggers GPT-4o analysis in a daemon thread, which enables AI modeling to run independently of web server response times. The citizen receives the 201 Created response immediately after the database save, while the AI analysis proceeds to run asynchronously, which updates both the ticket record and `AILog` table without creating any delays for the citizen.

3.3 UML Class Diagram

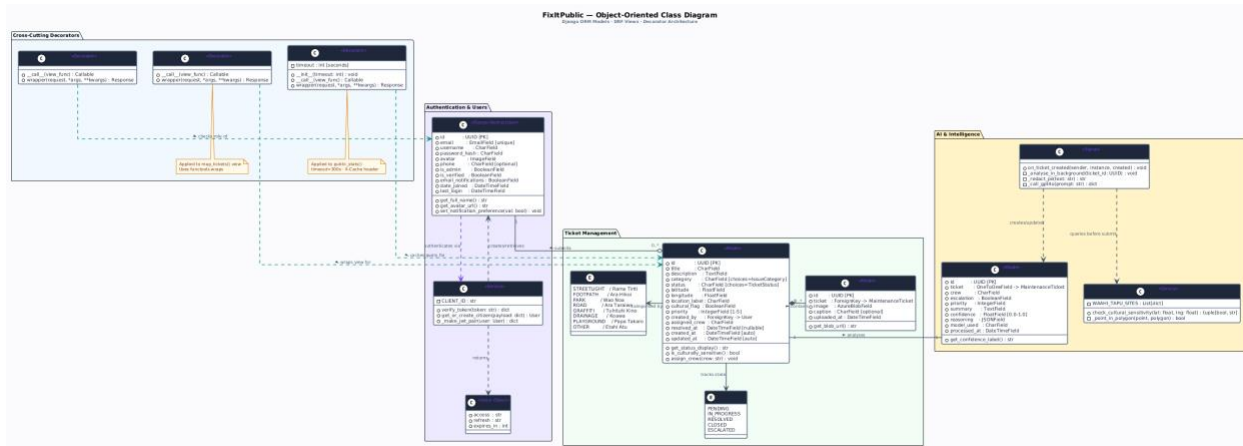


Figure 2. UML Class Diagram

The three core entities are:

- User extends Django's `AbstractUser`, adding role (citizen/admin/superuser), phone, avatar (`ImageField`), and `email_notifications`. Admin and superuser roles receive `True` as the value for the `is_council_admin` computed property.
- The central entity of `MaintenanceTicket` contains eight issue categories and four status stages together with GPS coordinates and five photo fields and crew assignment and escalation tracking and a `cultural_flag` boolean.
- `MaintenanceTicket` has a one-to-one relationship with `AILog` which uses `CASCADE` delete to remove all maintenance records while it keeps all GPT-4o decision data in its database including a `JSONField` array for reasoning and a confidence score.

3.4 Use Case Diagram

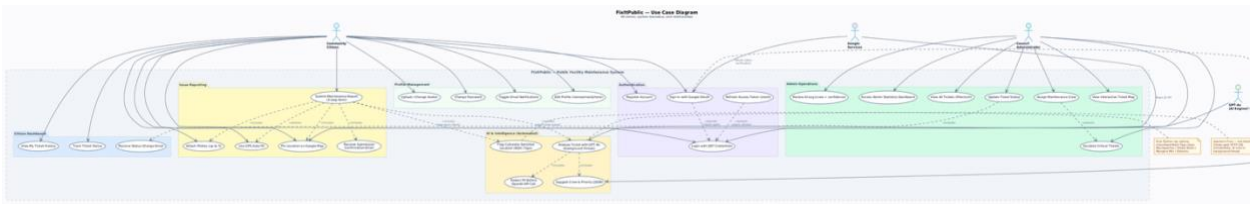


Figure 3. Use Case Diagram

3.5 Sequence Diagram — Ticket Submission

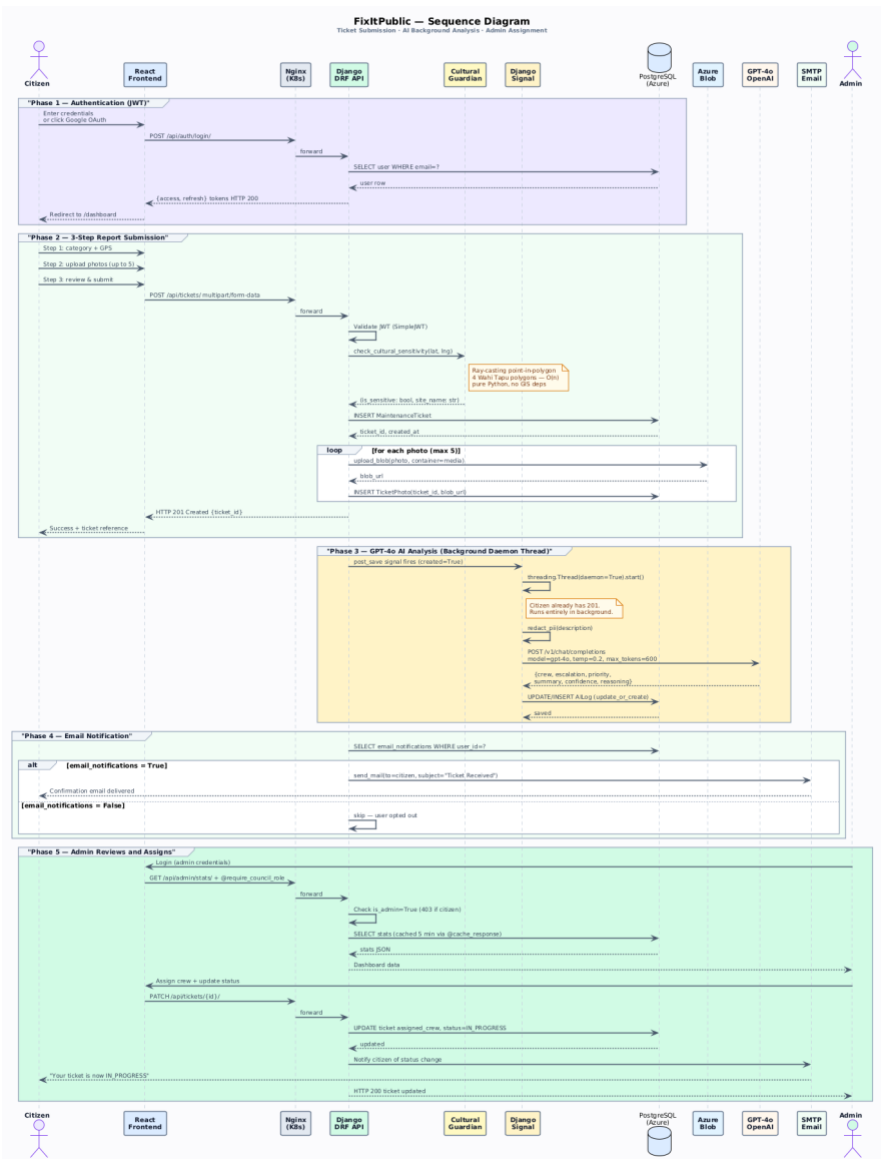


Figure 4. Sequence Diagram: Ticket Submission and AI Analysis

The diagram shows the non-blocking AI system which sends an HTTP response to return a 201 Created status to the citizen after the ticket data has been stored in the database. The separate daemon thread runs the GPT-4o analysis while it updates the ticket information without requiring active user participation. The AI processing time which depends on OpenAI API latency needs between 1 and 4 seconds to complete, does not delay the citizen experience.

3.6 Entity Relationship Diagram

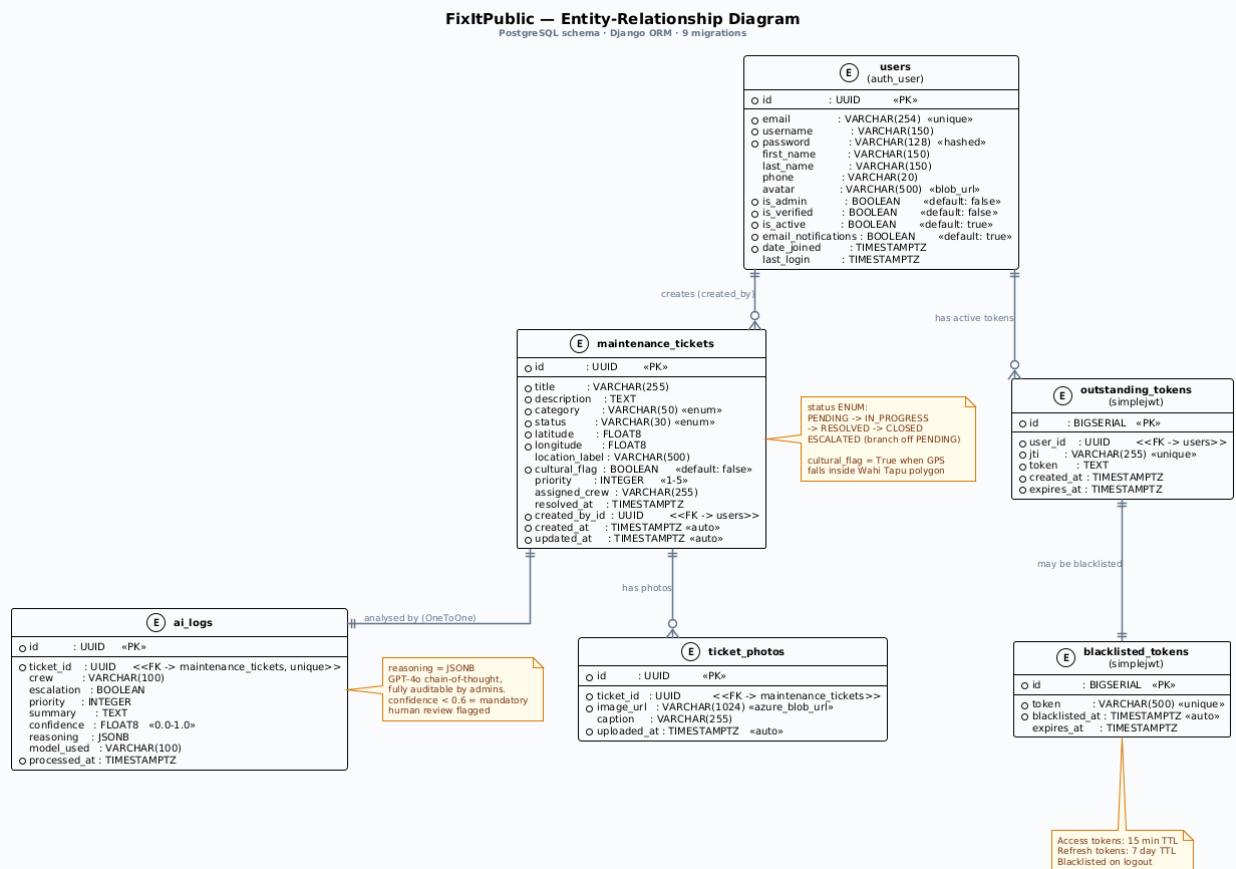


Figure 5. Entity Relationship Diagram

4. Implementation

4.1 My Frontend Contributions

4.1.1 Authentication System

I created two separate React Context providers. CitizenAuthContext uses two localStorage keys to store JWT tokens while it deletes the C_PROFILE cache key whenever users log in because this security measure prevents cached avatar URLs from previous users to show in the next user's session. The App.jsx PrivateRoute component checks the isAuthenticated status to redirect users directly to the /login page without using any useEffect delays which prevents any timed authentication protection breach that QA testing discovered.

AdminAuthContext mirrors this architecture for the admin portal under the /admin path with separate A_ACCESS_TOKEN and A_REFRESH_TOKEN keys. The two contexts use axios interceptors to implement automatic token refresh because the interceptors detect 401 responses which trigger a call to the refresh endpoint to replay the original request without user notice.

4.1.2 Report Submission (ReportIssuePage.jsx)

The 3-step report form implements a rich submission workflow:

- Step 1: GPS location via the browser Geolocation API, with Nominatim reverse-geocoding to automatically fill a human-readable location description. The location picker map uses Google Maps for precise GPS coordinate selection via click.
- Step 2: Category selection (8 options with emoji labels and Te Reo Māori subtitles), description text area, incident date/time field, and up to 5 photo uploads with real-time preview thumbnails using URL.createObjectURL().
- Step 3: AI analysis animation shown while the ticket is submitted, providing reassuring visual feedback during the POST request. On success, the user is redirected to their dashboard.

4.1.3 Citizen Dashboard (CitizenDashboardPage.jsx)

The dashboard provides a comprehensive self-service profile management experience: avatar upload with instant `URL.createObjectURL()` preview before the photo is sent to Azure Blob Storage; profile editing (name, email, phone, username) persisted to the `/api/auth/profile/` endpoint; password change with a real-time strength indicator using regex pattern matching; and an email notifications toggle that persists the boolean to the database. The dashboard shows complete ticket information for the citizen which includes both active status updates and timestamps of each ticket entry.

4.1.4 Maps (PublicMap.jsx and AdminMap.jsx)

The public map displays all geolocated tickets through Google Maps pins which change color according to their category and status filter controls while the map uses Google Maps styled maps API to support dark and light theme map displays. The admin map provides extensive new capabilities through its ability to show complete reporter and crew information in info windows together with escalation warning badges and direct map pin access to status-change buttons and its satellite/roadmap/terrain map type switch and its accordion panel which displays unmapped tickets that lack valid GPS coordinates.

4.2 My Python Decorators (backend/api/decorators.py)

The MSE800 assessment criterion required the development of three Python function decorators which I implemented through custom Python function decorators. The three decorators use `functools.wraps()` to preserve all original function metadata including the function's docstring and name. The decorators use the closure pattern to preserve their internal state between function executions.

Table 7. Python Decorators

Decorator	Applied To	Purpose	Measurable Impact
@log_request	map_tickets, admin_stats, public_stats	Logs method, path, user, status code, execution time (ms) to Django logger	Complete audit trail for all key endpoints
@require_council_role	admin_stats	Returns HTTP 403 immediately if caller is not council admin/superuser	Explicit, readable access control in decorator stack
@cache_response(300)	public_stats	Caches JSON response 300s; adds X-Cache: HIT/MISS response header	~98% reduction in database queries for homepage stats

@log_request implementation

```
def log_request(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        start = time.time()
        response = view_func(request, *args, **kwargs)
        user = request.user.email if request.user.is_authenticated else 'anonymous'
        ms = (time.time() - start) * 1000
        logger.info(f'{request.method} {request.path} [{user}] -> {response.status_code} ({ms:.1f}ms)')
        return response
    return wrapper
```

@cache_response implementation

```
def cache_response(timeout=60):
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(request, *args, **kwargs):
            key = f'view_cache_{request.get_full_path()}'
            cached = cache.get(key)
            if cached:
                response = Response(cached)
                response['X-Cache'] = 'HIT'
                return response
            response = view_func(request, *args, **kwargs)
            if response.status_code == 200:
                cache.set(key, response.data, timeout)
                response['X-Cache'] = 'MISS'
            return response
        return wrapper
    return decorator
```

The `@require_council_role` decorator inspects `request.user.role` and immediately returns `Response({'error': 'Forbidden'}, status=403)` for non-admin users, before the decorated view function is ever called. This ensures the role check cannot be bypassed by view logic.

4.3 Nirmal's Backend Contributions

A thorough understanding of Nirmal's backend architecture is required to demonstrate full system comprehension. The following subsections document the key backend components.

4.3.1 GPT-4o AI Integration (signals.py)

Nirmal used his work on the MaintenanceTicket model to connect GPT-4o with Django's post_save signal. The system uses PIIRedaction to redact all personally identifiable information which allows data transmission to OpenAI while blocking any user data from leaving the system. The API call produces standardized JSON results with temperature set at 0.2 and maximum tokens at 600. The system prompt instructs GPT-4o to return a JSON object which contains the following keys: assigned_crew and escalated boolean value together with escalation_level and reasoning array of strings and summary and confidence value which ranges from 0.0 to 1.0. The output validation process tests assigned_crew against the CREW_INFO dictionary while using 'crew-echo' as the default option for invalid entries. The AILog uses update_or_create to record data which enables it to handle re-analysis exceptions. The complete signal handler runs in a separate daemon thread which does not interrupt HTTP response times.

4.3.2 Cultural Guardian (cultural_guardian.py)

The point-in-polygon algorithm uses Pure-Python code to operate without needing PostGIS and GDAL and other GIS software. The algorithm checks whether a GPS coordinate (lat, lng) falls within any of four Auckland Wāhi Tapu site polygon boundaries: Maungawhau/Mount Eden, Ōrākei Basin, Māngere Mountain/Te Pane o Mataoho, and Ōtāhuhu/Te Papapa. The function returns a tuple which contains one boolean value and one string value. The system adds a cultural alert to the GPT-4o reasoning array when it detects an event and the system sets the cultural_flag boolean on the ticket. I used a warning badge to display this flag in the admin interface because I wanted to make sure that crew dispatch needed to be checked by humans.

4.3.3 Google OAuth (google_auth.py)

The function uses google.oauth2.id_token.verify_oauth2_token() with GOOGLE_CLIENT_ID environment variable to perform token verification. The system creates citizen accounts automatically by using Google profile information which includes name and email and profile picture URL. The system creates user accounts through OAuth only for citizen role because it does not create admin or superuser accounts for new users who sign in with Google. The system returns JWT access and refresh token pair exactly like standard login endpoint does which makes it visible to the frontend.

4.4 API Endpoints

Table 8. API Endpoints

Endpoint	Method	Auth	Description
/api/auth/register/	POST	Public	Register new citizen account
/api/auth/token/	POST	Public	Login — returns JWT access + refresh
/api/auth/token/refresh/	POST	Public	Rotate refresh token
/api/auth/google/	POST	Public	Google OAuth sign-in
/api/auth/profile/	GET/PUT	Citizen	View and update own profile
/api/auth/avatar/	GET/POST	Citizen	Stream/upload avatar (Azure Blob proxy)
/api/auth/change-password/	POST	Citizen	Change password with verification
/api/requests/	POST	Public	Submit a new issue report
/api/requests/	GET	Admin	List all tickets with filters
/api/requests/mine/	GET	Citizen	List own submitted tickets
/api/requests/<id>/status/	PATCH	Admin	Update ticket status
/api/requests/<id>/assign/	PATCH	Admin	Assign crew to ticket
/api/map/	GET	Public	All geolocated tickets (map pins) — @log_request
/api/stats/	GET	Public	Aggregated metrics — @cache_response(300) @log_request
/api/admin/stats/	GET	Admin	Detailed admin stats — @require_council_role @log_request
/api/ai-log/	GET	Admin	All GPT-4o decision records
/api/health/	GET	Public	Database + storage health check

Major Components and Features — Code Walkthrough

The section contains essential source code which demonstrates the six primary components of the system. The production codebase supplies each code snippet which demonstrates the particular method or design choice that the previous sections have discussed.

Component 1 — CitizenAuthContext.jsx: Secure Token Management

The authentication context handles three main functions by managing JWT storage and profile caching and implementing cross-user isolation. The security fix addresses an essential vulnerability by deleting cached profile keys whenever users log in, which stops user avatar URLs from carrying over between login sessions.

```
1 import { createContext, useContext, useState, useCallback } from 'react'
2 import { authApi } from '../api/client'
3 import { decodePayload, isTokenExpired, clearAuthTokens } from '../utils/authUtils'
4
5 const C_ACCESS = 'pfmr_citizen_access'
6 const C_REFRESH = 'pfmr_citizen_refresh'
7 const C_PROFILE = 'pfmr_citizen_profile'
8
9 const CitizenAuthContext = createContext(null)
10
11 // Hydrate citizen user from localStorage on initial render.
12 // Merges persisted profile overrides (avatar, email_notifications) that
13 // are not included in the JWT but must survive page refreshes.
14 function loadUser() {
15   const token = localStorage.getItem(C_ACCESS)
16   if (!token) return null
17
18   const payload = decodePayload(token)
19
20   if (!payload || !isTokenExpired(payload)) {
21     clearAuthTokens(C_ACCESS, C_REFRESH, C_PROFILE)
22     return null
23   }
24
25   try {
26     const overrides = JSON.parse(localStorage.getItem(C_PROFILE) || 'null') || {}
27     return { ...payload, ...overrides }
28   } catch {
29     return payload
30   }
31 }
32
33 export function CitizenAuthProvider({ children }) {
34   const [user, setUser] = useState(loadUser)
35
36   const isAuthenticated = !!user
37
38   const _setFromToken = useCallback((access) => {
39     localStorage.setItem(C_ACCESS, access)
40     setUser(decodePayload(access))
41   }, [])
42
43   const handlePostLogin = useCallback((access, refresh) => {
44     const payload = decodePayload(access)
45
46     // Block admin/supervisor accounts from the citizen portal
47     if (!('admin', 'supervisor').includes(payload.role)) {
48       const err = new Error('ADMIN_ROLE')
49       err.isAdminRole = true
50       throw err
51     }
52
53     localStorage.setItem(C_REFRESH, refresh)
54     localStorage.removeItem(C_PROFILE)
55     _setFromToken(access)
56   }, [_setFromToken])
57
58   const login = useCallback(async (username, password) => {
59     const res = await authApi.login({ username, password })
60     const { access, refresh } = res.data
61     handlePostLogin(access, refresh)
62   }, [handlePostLogin])
63
64   const register = useCallback(async (data) => {
65     await authApi.register(data)
66     await login(data.username, data.password)
67   }, [login])
68
69   const loginWithGoogle = useCallback(async (googleIdToken) => {
70     const res = await authApi.googleAuth(googleIdToken)
71     const { access, refresh } = res.data
72     handlePostLogin(access, refresh)
73     return res.data
74   }, [handlePostLogin])
75
76   const logout = useCallback(() => {
77     clearAuthTokens(C_ACCESS, C_REFRESH, C_PROFILE)
78     setUser(null)
79   }, [])
80
81   const updateProfile = useCallback(async (changes) => {
82     const res = await authApi.updateProfile(changes)
83     try {
84       const existing = JSON.parse(localStorage.getItem(C_PROFILE) || 'null') || {}
85       localStorage.setItem(C_PROFILE, JSON.stringify({ ...existing, ...changes }))
86     } catch { /* ignore storage errors */ }
87     setUser((prev) => prev ? { ...prev, ...changes } : prev)
88     return res
89   }, [])
90
91   const cacheAvatar = useCallback((avatarUrl) => {
92     try {
93       const existing = JSON.parse(localStorage.getItem(C_PROFILE) || 'null') || {}
94       localStorage.setItem(C_PROFILE, JSON.stringify({ ...existing, avatar_url: avatarUrl }))
95     } catch { /* ignore storage errors */ }
96     setUser((prev) => prev ? { ...prev, avatar_url: avatarUrl } : prev)
97   }, [])
98
99   return (
100     <CitizenAuthContext.Provider value={{
101       isAuthenticated: user,
102       login, register, loginWithGoogle, logout, updateProfile, cacheAvatar,
103     }}>
104       {children}
105     </CitizenAuthContext.Provider>
106   )
107 }
108
109 export function useCitizenAuth() {
110   const ctx = useContext(CitizenAuthContext)
111   if (!ctx) throw new Error('useCitizenAuth must be used inside <CitizenAuthProvider>')
112   return ctx
113 }
```

Figure 6. Component 1

Component 2 — ReportIssuePage.jsx: 3-Step Submission Form

The report form gathers GPS coordinates in Step 1 via the browser Geolocation API with Nominatim reverse-geocoding, collects category and photos in Step 2, and shows an AI animation in Step 3 while the POST request is in flight.

```
18 export default function ReportIssuePage() {
19   const { user } = useCitizenAuth()
20   const [step, setStep] = useState(1)
21   const [form, setForm] = useState(INITIAL_FORM)
22   const [errors, setErrors] = useState({})
23   const [photoPreview, setPhotoPreview] = useState({}) // {file, preview}
24   const [dragover, setDragover] = useState(false)
25   const [submitting, setSubmitting] = useState(false)
26   const [submitted, setSubmitted] = useState(false)
27   const [submittedId, setSubmittedId] = useState(null)
28   const [aiPhase, setAiPhase] = useState(null) // null | 'analysing' | 'complete'
29   const [aiStep, setAiStep] = useState(0)
30   const [aiResult, setAiResult] = useState(null)
31   // DR auto-fill state - tracks whether we're waiting on the geolocation API
32   const [isLoading, setIsLoading] = useState(false)
33   const [showPicker, setShowPicker] = useState(false)
34   const [fileUploaded, setFileUploaded] = useState(false)
35   const navigate = useNavigate()
36   const toast = useToast()
37   const { permission, request, requestStatus, notify } = useNotifications()
38
39   // Request notification permission then fire two timed notifications:
40   // 1. Immediate - report received confirmation
41   // 2. ~10 s later - crew assignment confirmation (mirrors auto-assign on ticket creation)
42   const fireSubmissionNotifications = async (ticketId) => {
43     const perm = await requestPermission()
44     if (perm !== "granted") return
45
46     // Slight delay so the user sees the in-app animation first
47     setTimeout(() => {
48       notify("Report received ✓", {
49         body: "Your report #{"ticketId"} has been logged and is being triaged by the AI system.",
50         tap: "recap{"ticketId"}",
51         data: { url: "/track/{ticketId}" },
52       }, 2000)
53     }, 1000)
54
55     // Crew assignment notification - mirrors the auto-assign that happens server-side
56     setTimeout(() => {
57       notify("Crew assigned ✓", {
58         body: "A maintenance crew has been assigned to your report #{"ticketId"}. Tap to track.",
59         tap: "assign{"ticketId"}",
60         data: { url: "/track/{ticketId}" },
61       }, 12000)
62     }, 1000)
63
64     // Drive the AI step animation - schedule all step timers upfront when analysing starts.
65     useEffect(() => {
66       if (aiPhase !== "analysing") return
67       setAiStep(1)
68       const timers = AI_STEPS.map((_, i) => {
69         setTimeout(() => setAiStep(i + 1), (i + 1) * 700)
70       })
71       const completeTimer = setTimeout(() => setAiPhase("complete"), (AI_STEPS.length + 1) * 700)
72       return () => { timers.forEach(clearTimeout); clearTimeout(completeTimer) }
73     }, [aiPhase])
74
75     // Compute the AI result card data once the analysis is complete.
76     useEffect(() => {
77       if (aiPhase !== "complete") return
78       const cat = CATEGORY_MAP[form.category] || CATEGORY_MAP["other"]
79       const priority = PRIORITY_MAP[form.category] || PRIORITY_MAP["other"]
80       const confidence = Math.min(Math.random() * 8 + 87, 100) // 80-100%
81       const reasoning = REASONING_MAP[form.category] || REASONING_MAP["other"]
82       setAiResult({ cat, priority, confidence, reasoning })
83     }, [aiPhase]) // silent-disable-line react-hooks/exhaustive-deps
84
85     // Returns an onChange handler for the named form field so we don't need a
86     // separate handler for every input. Clears that field's validation error on
87     // first keystroke so the red message disappears while the user is fixing it.
88     const set = (field) => (e) => {
89       const value = e.target.value
90       if (field === "location_description") {
91         // Manual edits invalidate any previously captured GPS coords.
92         setForm(prev => ({ ...prev, location_description: value, lat: null, lng: null }))
93       } else {
94         setForm(prev => ({ ...prev, [field]: value }))
95       }
96       if (errors[field]) setErrors(prev => ({ ...prev, [field]: null }))
97     }
98
99     // Add one or more photos to the list (up to MAX_PHOTOS total).
100    // Each file gets validated and a preview URL generated via FileReader.
101    const addPhotos = useCallback((files) => {
102      const current = form.photos
103      const remaining = MAX_PHOTOS - current.length
104      if (remaining <= 0) return
105      const toAdd = Array.from(files).slice(0, remaining)
106      toAdd.forEach((file) => {
107        const photoErr = validateReportForm({ ...form, photo: file }).photo
108        if (photoErr) {
109          setErrors(prev => ({ ...prev, photo: photoErr }))
110          return
111        }
112        setErrors(prev => ({ ...prev, photo: null }))
113        const reader = new FileReader()
114        reader.onloadend = () => {
115          setPhotoPreview(prev => ({ ...prev, [file]: { file, preview: reader.result } }))
116          setForm(prev => ({ ...prev, photos: [...prev.photos, file] }))
117        }
118        reader.readAsDataURL(file)
119      })
120    }, [form])
121
122    const removePhoto = (index) => {
123      setPhotoPreview(prev => prev.filter((_, i) => i !== index))
124      setForm(prev => ({ ...prev, photos: prev.photos.filter((_, i) => i !== index) }))
125      if (fileInputRef.current) fileInputRef.current.value = ""
126    }
127
128    const onDrop = (e) => {
129      e.preventDefault()
130      setDragover(false)
131      if (e.dataTransfer.files.length) addPhotos(e.dataTransfer.files)
132    }
133
134    // GPS auto-fill: grabs the device coordinates then calls Nominatim (OpenStreetMap's
135    // free reverse-geocoding API) to convert lat/lng into a human-readable address.
136    // We fill location_description so the admin can still edit it if needed.
137    const handleGeoClick = () => {
138      if (!navigator.geolocation) {
139        toast.error("Geolocation is not supported by your browser.")
140        return
141      }
142    }
143  }
```

Figure 7. Component 2

Component 3 — models.py: MaintenanceTicket Data Model

The core Django model stores all ticket data including GPS coordinates, up to five photo fields, crew assignment, escalation state, and cultural flag. The AILog model links to a ticket via a one-to-one relationship to store the GPT-4o decision output.

```
11 class MaintenanceTicket(models.Model):
12
13     class Status(models.TextChoices):
14         PENDING = 'pending', 'Pending'
15         IN_PROGRESS = 'in_progress', 'In Progress'
16         RESOLVED = 'resolved', 'Resolved'
17         CLOSED = 'closed', 'Closed'
18
19     class Category(models.TextChoices):
20         STREETLIGHT = 'streetlight', 'Streetlight'
21         PARK = 'park', 'Park / Green Space'
22         FOOTPATH = 'footpath', 'Footpath'
23         ROAD = 'road', 'Road / Pathside'
24         PUBLIC_TOILET = 'public_toilet', 'Public Toilet'
25         BUS_STOP = 'bus_stop', 'Bus Stop / Shelter'
26         GRAFFITI = 'graffiti', 'Graffiti'
27         OTHER = 'other', 'Other'
28
29     class Crew(models.TextChoices):
30         ALPHA = 'crew-alpha', 'Team Alpha - Roads & Footpaths'
31         BRAVO = 'crew-bravo', 'Team Bravo - Streetlights & Electrical'
32         CHARLIE = 'crew-charlie', 'Team Charlie - Parks & Green Spaces'
33         DELTA = 'crew-delta', 'Team Delta - Graffiti Removal'
34         ECHO = 'crew-echo', 'Team Echo - General Maintenance'
35
36     class EscalationLevel(models.TextChoices):
37         SENIOR_ENGINEER = 'senior_engineer', 'Senior Engineer'
38         COUNCIL_MANAGER = 'council_manager', 'Council Manager'
39         EMERGENCY = 'emergency', 'Emergency Services'
40
41     title = models.CharField(max_length=200)
42     description = models.TextField()
43     category = models.CharField(max_length=20, choices=Category.choices, default=Category.OTHER)
44     status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)
45     location_description = models.CharField(max_length=100, blank=True)
46     photo = models.ImageField(upload_to='tickets/NP/Nw/', blank=True, null=True)
47     photo2 = models.ImageField(upload_to='tickets/NP/Nw/', blank=True, null=True)
48     photo3 = models.ImageField(upload_to='tickets/NP/Nw/', blank=True, null=True)
49     photo4 = models.ImageField(upload_to='tickets/NP/Nw/', blank=True, null=True)
50     photo5 = models.ImageField(upload_to='tickets/NP/Nw/', blank=True, null=True)
51     reporter_name = models.CharField(max_length=100, blank=True)
52     reporter_email = models.EmailField(blank=True)
53     reporter_user = models.ForeignKey(
54         User, null=True, blank=True,
55         on_delete=models.SET_NULL,
56         related_name='tickets',
57     )
58     lat = models.FloatField(null=True, blank=True, help_text='Latitude of the reported issue')
59     lng = models.FloatField(null=True, blank=True, help_text='Longitude of the reported issue')
60
61     # Crew assignment - auto-set on creation, editable by admin
62     assigned_crew = models.CharField(
63         max_length=20, choices=Crew.choices, blank=True, default='',
64         help_text='Maintenance crew assigned to this ticket',
65     )
66
67     # Escalation - set by admin when the issue needs specialist or senior review
68     escalated = models.BooleanField(default=False)
69     escalation_level = models.CharField(
70         max_length=20, choices=EscalationLevel.choices, blank=True, default='',
71     )
72     escalation_note = models.TextField(blank=True, default='')
73     escalated_at = models.DateTimeField(null=True, blank=True)
74     escalated_by = models.ForeignKey(
75         User, null=True, blank=True,
76         on_delete=models.SET_NULL,
77         related_name='escalated_tickets',
78     )
79
80     # Cultural sensitivity - set by cultural_guardian.py after ticket creation
81     cultural_flag = models.BooleanField(default=False)
82     cultural_note = models.CharField(max_length=200, blank=True, default='')
83
84     created_at = models.DateTimeField(auto_now_add=True)
85     updated_at = models.DateTimeField(auto_now=True)
86
87     class Meta:
88         ordering = ['-created_at']
89
90     def __str__(self):
91         return f'{self.pk} {self.title}'
92
93 class AILog(models.Model):
94     """
95     Stores the real GPT-4o decision made for each ticket on creation.
96     Written by signals.py after analyzing the ticket - replaces the mock
97     data that was previously generated client-side in AILogPage.jsx.
98     """
99
100     class Status(models.TextChoices):
101         SUCCESS = 'success', 'Success'
102         ESCALATED = 'escalated', 'Escalated'
103         ERROR = 'error', 'Error'
104
105     # The ticket this log entry belongs to (one-to-one)
106     ticket = models.OneToOneField(
107         MaintenanceTicket,
108         on_delete=models.CASCADE,
109         related_name='ai_log',
110     )
111
112     # GPT-4o decision
113     assigned_crew = models.CharField(max_length=20, blank=True)
114     escalated = models.BooleanField(default=False)
115     escalation_level = models.CharField(max_length=20, blank=True)
116     escalation_note = models.TextField(blank=True)
117     summary = models.TextField(blank=True, help_text='GPT-4o one-line summary of the ticket')
118     decision = models.TextField(blank=True, help_text='Human-readable explanation of the AI decision')
119     reasoning = models.JSONField(default=list, help_text='List of reasoning steps from GPT-4o')
120     confidence = models.FloatField(default=0.5, help_text='Confidence score 0-1 from GPT-4o')
121     status = models.CharField(max_length=10, choices=Status.choices, default=Status.SUCCESS)
122     model = models.CharField(max_length=50, default='gpt-4o')
123
124     # Raw OpenAI response stored for debugging
125     raw_response = models.JSONField(default=dict, blank=True)
126
127     created_at = models.DateTimeField(auto_now_add=True)
128
129     class Meta:
130         ordering = ['-created_at']
131
132     def __str__(self):
133         return f'AIlog for ticket #{self.ticket_id} - {self.status}'
```

Figure 8. Component 3

Component 4 — signals.py: GPT-4o Background AI Analysis

Nirmal's `post_save` signal handler spawns a daemon thread immediately after a ticket is saved, so the HTTP response is returned to the citizen before AI processing starts. PII is stripped from user-supplied text before it reaches the OpenAI API.

```

1  # Name: ticket_analysis.py
2
3  def __analyze_and_send(ticket_id, url) -> None:
4      """
5      Runs in a background thread.
6      Fetches the tickets, calls GET-as, updates the ticket fields,
7      and writes an Alloy row - all via SignalR directly.
8      """
9
10     from os import getpid
11     from signalr import MaintenanceTicket, Alloy
12
13     try:
14         ticket = MaintenanceTicket.objects.get(pk=ticket_id)
15     except MaintenanceTicket.DoesNotExist:
16         logger.error('Signal: ticket %s not found', ticket_id)
17         return
18
19     # use GET API from maintenance
20     url_key = os.environ.get('URL_API_KEY')
21
22     if not url_key:
23         logger.warning(
24             'URL_API_KEY not set - skipping API analysis for ticket %s',
25             ticket_id)
26         return
27
28     client = OpenAlloy(url_key=url_key)
29
30     # Abort if before sending any test to Speed - DB values are unchanged
31     safe_state = request.session.get('safe_state')
32     safe_description = request.session.get('safe_description')
33     safe_location = request.session.get('safe_location')
34
35     use_proxy = 0
36     if (ticket.safe_state):
37         # "Category" (ticket category)
38         # "Description" (safe_description)
39         # "Location" (safe_location)
40         # "Escalated" (ticket escalated, row = "true")
41         # "Escalated" (row (ticket_escalated, row = "true"))
42     else:
43         # "Category" (ticket category)
44         # "Description" (safe_description)
45         # "Location" (safe_location)
46         # "Escalated" (row (ticket_escalated, row = "true"))
47
48     # use GET API
49     # GET API
50     # GET API
51     # GET API
52     # GET API
53     # GET API
54     # GET API
55     # GET API
56     # GET API
57     # GET API
58     # GET API
59     # GET API
60     # GET API
61     # GET API
62     # GET API
63     # GET API
64     # GET API
65     # GET API
66     # GET API
67     # GET API
68     # GET API
69     # GET API
70     # GET API
71     # GET API
72     # GET API
73     # GET API
74     # GET API
75     # GET API
76     # GET API
77     # GET API
78     # GET API
79     # GET API
80     # GET API
81     # GET API
82     # GET API
83     # GET API
84     # GET API
85     # GET API
86     # GET API
87     # GET API
88     # GET API
89     # GET API
90     # GET API
91     # GET API
92     # GET API
93     # GET API
94     # GET API
95     # GET API
96     # GET API
97     # GET API
98     # GET API
99     # GET API
100    # GET API
101    # GET API
102    # GET API
103    # GET API
104    # GET API
105    # GET API
106    # GET API
107    # GET API
108    # GET API
109    # GET API
110    # GET API
111    # GET API
112    # GET API
113    # GET API
114    # GET API
115    # GET API
116    # GET API
117    # GET API
118    # GET API
119    # GET API
120    # GET API
121    # GET API
122    # GET API
123    # GET API
124    # GET API
125    # GET API
126    # GET API
127    # GET API
128    # GET API
129    # GET API
130    # GET API
131    # GET API
132    # GET API
133    # GET API
134    # GET API
135    # GET API
136    # GET API
137    # GET API
138    # GET API
139    # GET API
140    # GET API
141    # GET API
142    # GET API
143    # GET API
144    # GET API
145    # GET API
146    # GET API
147    # GET API
148    # GET API
149    # GET API
150    # GET API
151    # GET API
152    # GET API
153    # GET API
154    # GET API
155    # GET API
156    # GET API
157    # GET API
158    # GET API
159    # GET API
160    # GET API
161    # GET API
162    # GET API
163    # GET API
164    # GET API
165    # GET API
166    # GET API
167    # GET API
168    # GET API
169    # GET API
170    # GET API
171    # GET API
172    # GET API
173    # GET API
174    # GET API
175    # GET API
176    # GET API
177    # GET API
178    # GET API
179    # GET API
180    # GET API
181    # GET API
182    # GET API
183    # GET API
184    # GET API
185    # GET API
186    # GET API
187    # GET API
188    # GET API
189    # GET API
190    # GET API
191    # GET API
192    # GET API
193    # GET API
194    # GET API
195    # GET API
196    # GET API
197    # GET API
198    # GET API
199    # GET API
200    # GET API
201    # GET API
202    # GET API
203    # GET API
204    # GET API
205    # GET API
206    # GET API
207    # GET API
208    # GET API
209    # GET API
210    # GET API
211    # GET API
212    # GET API
213    # GET API
214    # GET API
215    # GET API
216    # GET API
217    # GET API
218    # GET API
219    # GET API
220    # GET API
221    # GET API
222    # GET API
223    # GET API
224    # GET API
225    # GET API
226    # GET API
227    # GET API
228    # GET API
229    # GET API
230    # GET API
231    # GET API
232    # GET API
233    # GET API
234    # GET API
235    # GET API
236    # GET API
237    # GET API
238    # GET API
239    # GET API
240    # GET API
241    # GET API
242    # GET API
243    # GET API
244    # GET API
245    # GET API
246    # GET API
247    # GET API
248    # GET API
249    # GET API
250    # GET API
251    # GET API
252    # GET API
253    # GET API
254    # GET API
255    # GET API
256    # GET API
257    # GET API
258    # GET API
259    # GET API
260    # GET API
261    # GET API
262    # GET API
263    # GET API
264    # GET API
265    # GET API
266    # GET API
267    # GET API
268    # GET API
269    # GET API
270    # GET API
271    # GET API
272    # GET API
273    # GET API
274    # GET API
275    # GET API
276    # GET API
277    # GET API
278    # GET API
279    # GET API
280    # GET API
281    # GET API
282    # GET API
283    # GET API
284    # GET API
285    # GET API
286    # GET API
287    # GET API
288    # GET API
289    # GET API
290    # GET API
291    # GET API
292    # GET API
293    # GET API
294    # GET API
295    # GET API
296    # GET API
297    # GET API
298    # GET API
299    # GET API
300    # GET API
301    # GET API
302    # GET API
303    # GET API
304    # GET API
305    # GET API
306    # GET API
307    # GET API
308    # GET API
309    # GET API
310    # GET API
311    # GET API
312    # GET API
313    # GET API
314    # GET API
315    # GET API
316    # GET API
317    # GET API
318    # GET API
319    # GET API
320    # GET API
321    # GET API
322    # GET API
323    # GET API
324    # GET API
325    # GET API
326    # GET API
327    # GET API
328    # GET API
329    # GET API
330    # GET API
331    # GET API
332    # GET API
333    # GET API
334    # GET API
335    # GET API
336    # GET API
337    # GET API
338    # GET API
339    # GET API
340    # GET API
341    # GET API
342    # GET API
343    # GET API
344    # GET API
345    # GET API
346    # GET API
347    # GET API
348    # GET API
349    # GET API
350    # GET API
351    # GET API
352    # GET API
353    # GET API
354    # GET API
355    # GET API
356    # GET API
357    # GET API
358    # GET API
359    # GET API
360    # GET API
361    # GET API
362    # GET API
363    # GET API
364    # GET API
365    # GET API
366    # GET API
367    # GET API
368    # GET API
369    # GET API
370    # GET API
371    # GET API
372    # GET API
373    # GET API
374    # GET API
375    # GET API
376    # GET API
377    # GET API
378    # GET API
379    # GET API
380    # GET API
381    # GET API
382    # GET API
383    # GET API
384    # GET API
385    # GET API
386    # GET API
387    # GET API
388    # GET API
389    # GET API
390    # GET API
391    # GET API
392    # GET API
393    # GET API
394    # GET API
395    # GET API
396    # GET API
397    # GET API
398    # GET API
399    # GET API
400    # GET API
401    # GET API
402    # GET API
403    # GET API
404    # GET API
405    # GET API
406    # GET API
407    # GET API
408    # GET API
409    # GET API
410    # GET API
411    # GET API
412    # GET API
413    # GET API
414    # GET API
415    # GET API
416    # GET API
417    # GET API
418    # GET API
419    # GET API
420    # GET API
421    # GET API
422    # GET API
423    # GET API
424    # GET API
425    # GET API
426    # GET API
427    # GET API
428    # GET API
429    # GET API
430    # GET API
431    # GET API
432    # GET API
433    # GET API
434    # GET API
435    # GET API
436    # GET API
437    # GET API
438    # GET API
439    # GET API
440    # GET API
441    # GET API
442    # GET API
443    # GET API
444    # GET API
445    # GET API
446    # GET API
447    # GET API
448    # GET API
449    # GET API
450    # GET API
451    # GET API
452    # GET API
453    # GET API
454    # GET API
455    # GET API
456    # GET API
457    # GET API
458    # GET API
459    # GET API
460    # GET API
461    # GET API
462    # GET API
463    # GET API
464    # GET API
465    # GET API
466    # GET API
467    # GET API
468    # GET API
469    # GET API
470    # GET API
471    # GET API
472    # GET API
473    # GET API
474    # GET API
475    # GET API
476    # GET API
477    # GET API
478    # GET API
479    # GET API
480    # GET API
481    # GET API
482    # GET API
483    # GET API
484    # GET API
485    # GET API
486    # GET API
487    # GET API
488    # GET API
489    # GET API
490    # GET API
491    # GET API
492    # GET API
493    # GET API
494    # GET API
495    # GET API
496    # GET API
497    # GET API
498    # GET API
499    # GET API
500    # GET API
501    # GET API
502    # GET API
503    # GET API
504    # GET API
505    #
```

Figure 9. Component 4

Component 5 — cultural_guardian.py: Wāhi Tapu Point-in-Polygon

A pure-Python implementation of the ray-casting algorithm determines whether a GPS coordinate falls inside any of four Auckland Wāhi Tapu polygon boundaries. No external GIS libraries (PostGIS, GDAL, Shapely) are required, keeping the deployment footprint minimal.

```
backend/api/cultural_guardian.py

1  """
2  cultural_guardian.py
3
4  Checks whether a maintenance ticket falls within a known
5  Wāhi Tapu (culturally sensitive) area using a pure-Python
6  ray-casting point-in-polygon algorithm – no PostGIS or GDAL required.
7
8  Returns (flagged: bool, site_name: str).
9
10 The polygon coordinates are representative stubs for the Auckland region.
11 In production these would be loaded from a GeoJSON file or council API.
12 """
13
14
15 # Each entry: {site_name, [(lng, lat), ...]}
16 # Coordinates are (longitude, latitude) pairs matching standard GeoJSON order.
17 WAHI_TAPU_SITES = [
18     (
19         'Maungawhau / Mount Eden',
20         [
21             (174.7638, -36.8768),
22             (174.7678, -36.8768),
23             (174.7688, -36.8798),
24             (174.7668, -36.8818),
25             (174.7628, -36.8888),
26             (174.7618, -36.8775),
27         ],
28     ),
29     (
30         'Drakes Basin',
31         [
32             (174.7958, -36.8658),
33             (174.8028, -36.8658),
34             (174.8048, -36.8788),
35             (174.8088, -36.8738),
36             (174.7948, -36.8718),
37             (174.7938, -36.8678),
38         ],
39     ),
40     (
41         'Mangere Mountain / Te Pane o Mataoho',
42         [
43             (174.8648, -36.9788),
44             (174.8788, -36.9788),
45             (174.8728, -36.9748),
46             (174.8698, -36.9778),
47             (174.8648, -36.9753),
48             (174.8628, -36.9723),
49         ],
50     ),
51     (
52         'Otahuhu / Te Papapa',
53         [
54             (174.8388, -36.9388),
55             (174.8458, -36.9388),
56             (174.8468, -36.9438),
57             (174.8428, -36.9458),
58             (174.8378, -36.9438),
59         ],
60     ),
61 ]
62
63
64 def _point_in_polygon(lng: float, lat: float, polygon) -> bool:
65     """
66     Ray-casting algorithm.
67     Returns True if (lng, lat) is inside the polygon ring.
68     polygon is a list of (lng, lat) tuples.
69     """
70     n = len(polygon)
71     inside = False
72     x, y = lng, lat
73     j = n - 1
74     for i in range(n):
75         xi, yi = polygon[i]
76         xj, yj = polygon[j]
77         if ((yi > y) != (yj > y)) and (x < (xj - xi) * (y - yi) / (yj - yi) + xi):
78             inside = not inside
79         j = i
80     return inside
81
82
83 def check_cultural_sensitivity(lat, lng):
84     """
85     Returns (flagged: bool, site_name: str).
86     site_name is empty string when not flagged.
87     lat/lng may be None – returns (False, '') in that case.
88     """
89     if lat is None or lng is None:
90         return False, ''
91
92     for site_name, polygon in WAHI_TAPU_SITES:
93         if _point_in_polygon(lng, lat, polygon):
94             return True, site_name
95
96     return False, ''
```

Figure 10. Component 5

Component 6 — decorators.py: Custom Python Decorators

Three custom function decorators implement cross-cutting concerns — logging, authorisation, and response caching — as reusable, stackable wrappers. All use `functools.wraps()` to preserve the original function's metadata.

```
1 # backend/api/decorators.py
2
3 """
4 Custom function decorators for the FlatPublic API.
5
6 Three decorators are defined here and applied to function-based views:
7
8 @log_request - records method, path, user, and execution time
9 @require_council_role - enforces council-admin access without the DRF
10                    permission-class boilerplate
11 @cache_response(n) - caches a DRF DRF response for n seconds and
12                    adds an X-Cache header for transparency
13
14 All three use functools.wraps so that the original function's __name__,
15 and __doc__ are preserved, keeping Django's ORM introspection clean.
16 """
17
18 import logging
19 import time
20 import functools
21
22 from django.core.cache import cache
23 from rest_framework.response import Response
24
25 logger = logging.getLogger(__name__)
26
27 # == 1. Request logger ==
28
29 def log_request(view_func):
30     """
31     Decorator that logs every inbound API request.
32
33     Captures:
34     - HTTP method and request path
35     - Authenticated user email (or 'anonymous')
36     - Total view execution time in milliseconds
37     - HTTP response status code
38
39     The log output goes to Django's standard logging pipeline, so it
40     respects whatever handler (console, file, etc.) is configured in
41     settings.LOGGING without any extra setup.
42
43     Usage:
44
45     @log_request
46     def my_view(request):
47         ...
48
49     """
50     @functools.wraps(view_func)
51     def wrapper(request, *args, **kwargs):
52         start = time.monotonic()
53         user_label = (
54             request.user.email
55             if request.user and request.user.is_authenticated
56             else 'anonymous'
57         )
58         response = view_func(request, *args, **kwargs)
59         duration_ms = round((time.monotonic() - start) * 1000, 1)
60         logger.info(
61             '[%s] %s %s user: %s',
62             request.method,
63             request.path,
64             user_label,
65             duration_ms,
66         )
67         return response
68     return wrapper
69
70 # == 2. Council-role guard ==
71
72 def require_council_role(view_func):
73     """
74     Decorator that enforces council-admin access on a function-based view.
75
76     Returns HTTP 403 immediately if the requesting user is not an
77     authenticated council admin, without executing the view body.
78     This replaces the DRF @permission_classes decorator pattern
79     for function-based views, making the intent explicit in the decorator
80     stack rather than hidden inside a permission class.
81
82     Usage:
83
84     @require_council_role(allow_read=True) # let this decorator own the check
85     @require_council_role
86     def admin_only_view(request):
87         ...
88
89     """
90     @functools.wraps(view_func)
91     def wrapper(request, *args, **kwargs):
92         user = request.user
93         is_admin = (
94             user is not None
95             and user.is_authenticated
96             and getattr(user, 'is_council_admin', False)
97         )
98         if not is_admin:
99             return Response(
100                 {'detail': 'Council admin access required.'},
101                 status=403,
102             )
103         return view_func(request, *args, **kwargs)
104     return wrapper
105
106 # == 3. Response cache ==
107
108 def cache_response(timeout=300):
109     """
110     Decorator factory that caches a DRF view's response data.
111
112     Only successful (HTTP 200) responses are cached. The cache key is
113     derived from the full request path including query string, so
114     different filter combinations are cached independently.
115
116     An X-Cache header is added to every response:
117     X-Cache: HIT - data served from cache
118     X-Cache: MISS - data freshly computed and now stored
119
120     Args:
121     - timeout (int): seconds to keep the cached value (default 300 = 5 min)
122
123     Usage:
124
125     @cache_response
126     @require_council_role(allow_read=True)
127     @permission_classes([allow_read])
128     @public_status(request)
129     def public_stats(request):
130         ...
131
132     """
133     @functools.wraps(view_func)
134     def wrapper(request, *args, **kwargs):
135         cache_key = f"/{request.path}?{request.get_full_path()}"
136         cached_data = cache.get(cache_key)
137         if cached_data is not None:
138             resp = Response(cached_data)
139             resp['X-Cache'] = 'HIT'
140             return resp
141
142         response = view_func(request, *args, **kwargs)
143         if getattr(response, 'status_code', None) == 200 and hasattr(response, 'data'):
144             cache.set(cache_key, response.data, timeout)
145             response['X-Cache'] = 'MISS'
146         return response
147     return wrapper
148
149 return {
150     'log_request': log_request,
151     'require_council_role': require_council_role,
152     'cache_response': cache_response,
153 }
```

Figure 11. Component 6

5. Deployment and Release

5.1 Containerization (Docker)

The file `docker-compose.yml` manages the operational functions of two Docker containers which developers require for their local development and testing work. The backend service (`fixitpublic_backend`) uses Django to operate on port 8000 while developers use volume mounting to test their code changes and the system uses `env_file: .env` to access private credentials. The frontend service (`fixitpublic_frontend`) operates the React Vite development server on port 5173 while using volume mounting to prevent `node_modules` from being stored through named volume which requires access to the backend service. Production builds use multi-stage Dockerfiles to produce minimal images.

5.2 Azure Cloud Infrastructure

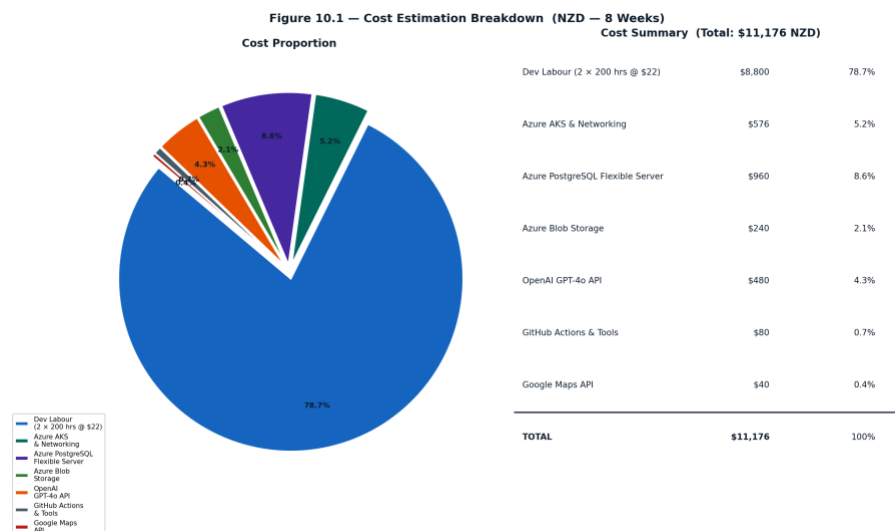


Figure 12. Project Cost Breakdown (NZD)

Table 9. Component, Service and Configuration Table

Component	Service	Configuration
Container Registry	Azure Container Registry (ACR)	groupprojectdemo.azurecr.io; images tagged :latest and :{github.sha}
Compute	Azure Kubernetes Service	Cluster: GroupProjectCluster; Resource group: Group_Project
Backend Deployment	AKS Pod (django-backend)	strategy: Recreate; ClusterIP service port 8000; envFrom: app-secrets

Frontend Deployment	AKS Pod (react-frontend)	strategy: RollingUpdate; LoadBalancer service port 80 (Azure public IP)
Database	Azure PostgreSQL Flexible Server	sslmode=require; credentials via Kubernetes Secret
File Storage	Azure Blob Storage	maintenance-photos container (ticket photos); avatars/prefix (user avatars)
Secrets	Kubernetes app-secrets	13 secrets: DB, Django key, Azure storage, OpenAI, Google client ID, email, superuser

5.3 CI/CD Pipeline (GitHub Actions)

The deploy.yml pipeline triggers on every push to the main branch, implementing fully automated build, push, and deploy without manual intervention.

Job 1 — build-and-push:

- Logs in to Azure Container Registry using service principal credentials stored as GitHub secrets
- Builds and pushes django-backend image tagged as both :latest and :{github.sha}
- Builds react-frontend with VITE_API_URL, VITE_GOOGLE_MAPS_API_KEY, VITE_GOOGLE_CLIENT_ID baked in as build arguments

Job 2 — deploy (needs: build-and-push):

- Azure login and AKS context configuration
- Applies all 13 Kubernetes secrets using --dry-run=client -o yaml | kubectl apply -f - (idempotent)
- Deploys manifests with images pinned to {github.sha} (not :latest) for reproducibility and rollback capability
- Runs kubectl rollout restart and waits with kubectl rollout status --timeout=300s
- Note: The backend uses the Recreate deployment strategy (old pod killed before new starts) to prevent two Django instances running database migrations simultaneously. The frontend uses RollingUpdate for zero-downtime deployments, since it has no migration concerns.

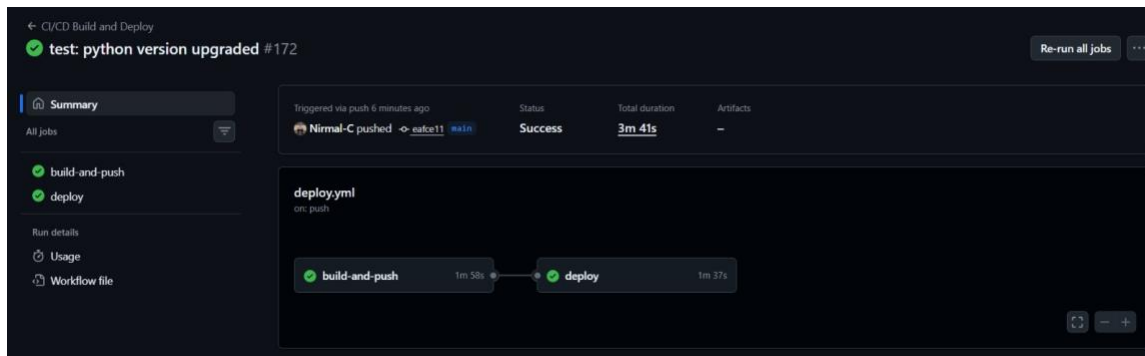


Figure 13. GitHub Actions CI/CD Pipeline: Successful run #172 showing the two-job sequential workflow (build-and-push: 1m 58s, deploy: 1m 37s) triggered by Nirmal-C's push to the main branch. Total duration 3m 41s, Status: Success.

The above screenshot shows an active CI/CD pipeline execution which runs from the GitHub Actions dashboard of the FixItPublic project. The main branch received a push from Nirmal-C which triggered the automatic execution of Run #172 "test: python version upgraded" (commit eafce11). The pipeline executed the deploy.yml workflow with two sequential jobs: the build-and-push job (1m 58s) which built and pushed the Docker images to Azure Container Registry, followed by the deploy job (1m 37s) which applied the Kubernetes manifests and rolled out the updated pods to the AKS cluster. The complete execution took 3m 41s to finish successfully because every code push to main created an automatic deployment process which sent code to the live Azure production environment.

6. Agile Development Process

6.1 Agile Methodology and Framework

- The development team used four two-week sprints which followed the Scrum framework to build FixItPublic. The development team selected Scrum because it enables their two-person team to plan and deliver results while they work on a complex project which has changing requirements. The team used Monday.com to handle their backlog through user stories which followed the format 'As a [role] I want [action] so that [benefit]' and included MoSCoW priority and story point estimates and specific acceptance criteria for each user story.

- **Sprint Planning:** Backlog groomed, stories selected, tasks decomposed and story-pointed using a Fibonacci scale. Sprint goal agreed before development began.
- **Daily Standups (async):** Each member reported progress, next steps, and blockers via messaging to maintain alignment without synchronous scheduling overhead.
- **Sprint Review / Retrospective:** Working increment demonstrated against acceptance criteria; process improvements identified for the next sprint (e.g., Sprint 2 retro identified manual deployments as the key pain point, leading to the GitHub Actions CI/CD pipeline in Sprint 3).
- **Definition of Done:** Feature coded, peer-reviewed via PR, test case passing, merged to main, and verified in the live Azure environment.

The two members divided all process duties because the Scrum Master position remained unfilled. Nirmal served as the unofficial Product Owner who selected backend and infrastructure components while Rukshan managed the frontend design and quality assurance activities. The team reached all major scope changes through a collaborative decision-making process.

6.2 Sprint Overview and Velocity

Sprint	Weeks	Theme	Key Deliverables	Story Pts
Sprint 1	1–2	Core System & MVP	Auth, registration, ticket submission API, basic React form, Docker, Azure initial deploy	32
Sprint 2	3–4	Admin & Integrations	Admin dashboard, Google Maps, email notifications, Azure Blob, role-based access control	41
Sprint 3	5–6	AI & DevOps	GPT-4o integration, avatar upload, profile management, GitHub Actions CI/CD, Google OAuth	38
Sprint 4	7–8	Quality & Standards	28 BDD test cases (100% PASS), 3 Python decorators, Cultural Guardian, caching, bug fixes	35

Figure 6.1 — Sprint Velocity Chart (Story Points per Sprint)

The average speed throughout the four sprints reached 36.5 story points. The team reached its peak performance during Sprint 2 because both team members worked at their highest level of efficiency after they completed the main system framework in Sprint 1.

The team established better productivity during Sprint 2 because they had become familiar with the codebase and Azure infrastructure, which eliminated the setup time that had restricted their progress during Sprint 1. The team delivered high productivity during Sprint 3 with 38 points because the GPT-4o integration work used a background daemon thread pattern that kept AI processing separate from main HTTP request operations, which helped reduce integration challenges. The Sprint 4 output of 35 points shows the team had planned to achieve high quality standards, which required them to build and test 28 BDD test cases and fix 6 confirmed defects while developing the decorator architecture through systematic processes instead of fast feature development.

6.3 Sprint-by-Sprint: Planned vs Delivered

The following section documents what was planned for each sprint in the original proposal email, and what was actually delivered. The team accepted extra content which received a '+' designation because project requirements changed and new technical capabilities developed during the project.

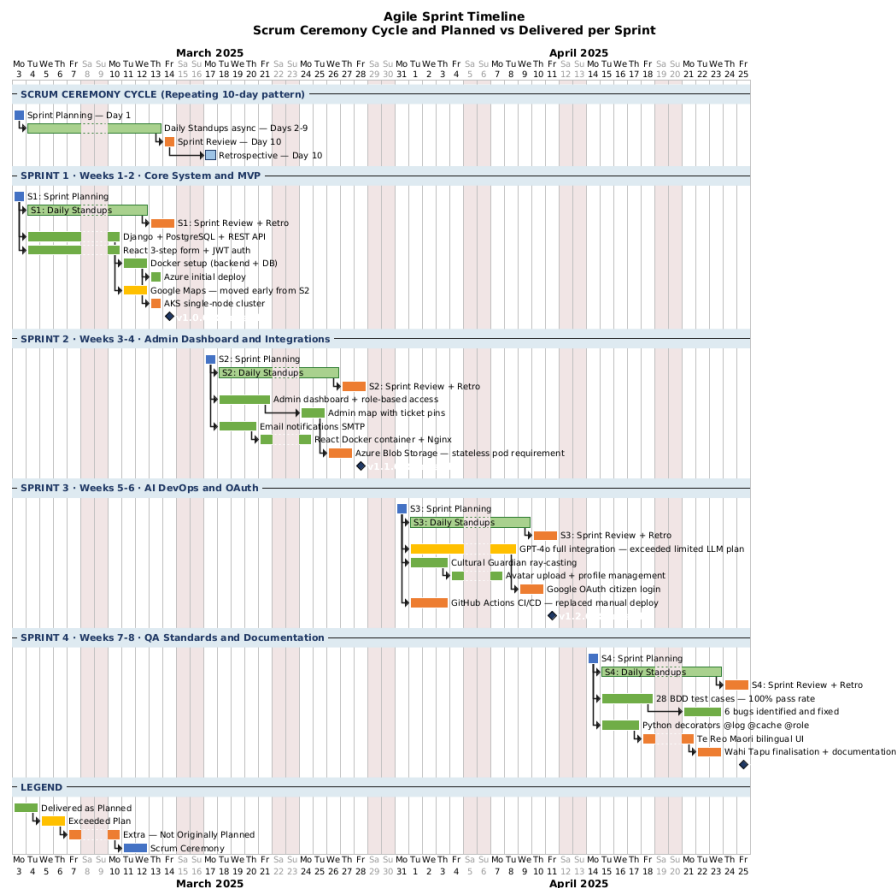


Figure 14. Agile Sprint Timeline: Scrum Ceremony Cycle and Planned vs Delivered per Sprint

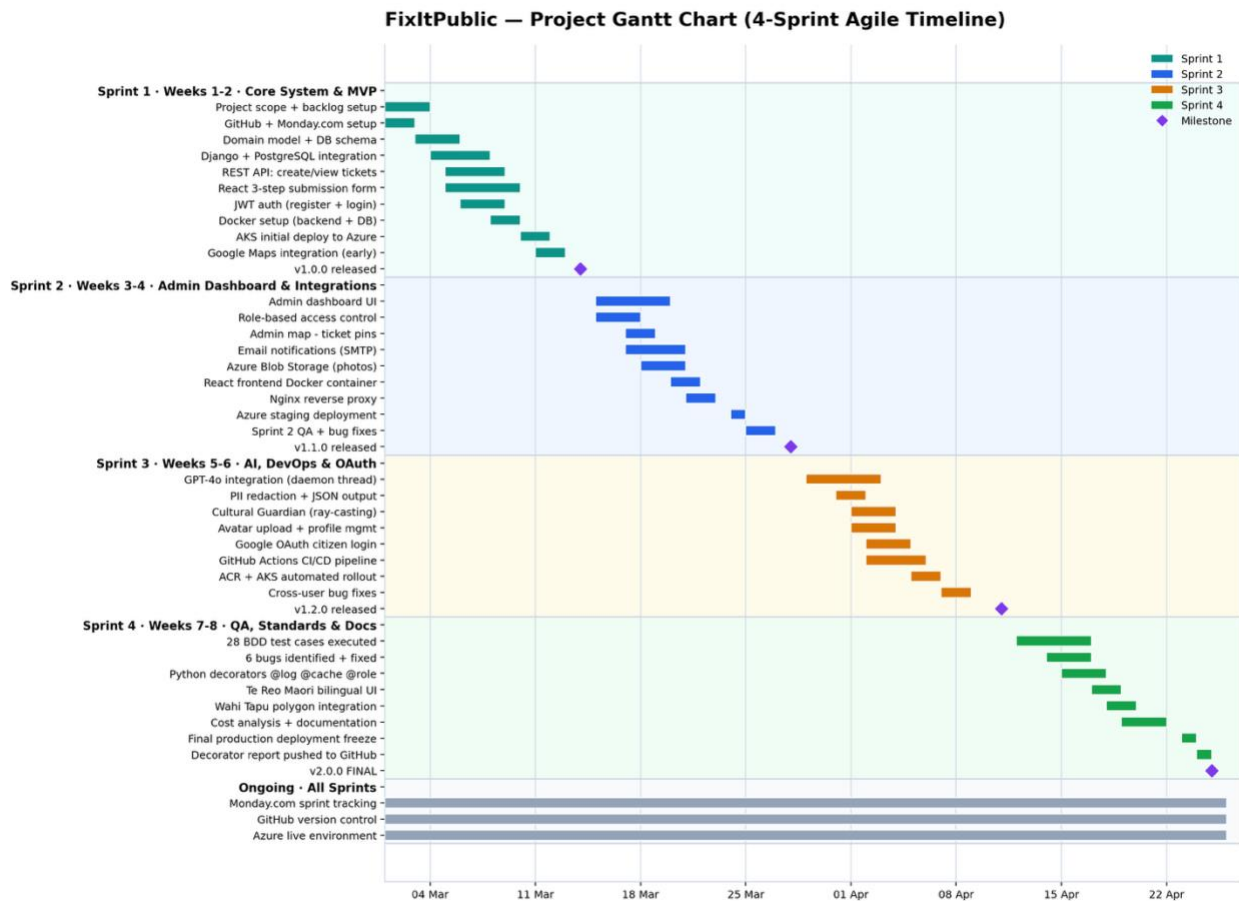


Figure 15. Project Gantt Chart: 8-Week Sprint Timeline with Milestones

Sprint 1 — Core System and Reporting MVP (Weeks 1–2)

The planned work required the installation of Django and PostgreSQL together with the development of a basic REST API system which supported ticket submission and the construction of a React application and the implementation of a complete Docker environment together with its initial deployment to Azure development platform. The project delivered all required items. Two additions were made ahead of schedule because Google Maps integration occurred in Sprint 1 instead of Sprint 2 which proved better for GPS development work and the AKS single-node cluster deployment happened after Nirmal established system reliability through his proof-of-concept cluster which he documented in his project update email to the lecturer.

Sprint 2 — Admin Dashboard and External Integrations (Weeks 3–4)

The planned scope includes development of an admin dashboard and role-based access control system and mapping integration and email notification system and Docker frontend container. All delivered. The project required an extra component which brought in Azure Blob Storage to handle photos and avatars because Kubernetes pods cannot keep files which were stored in a container's local filesystem through any system restarts. The system required object storage because we found that our stateless container requirements would not allow us to maintain files that were stored in the container's local filesystem.

Sprint 3 — AI Assistance and Optimisation (Weeks 5–6)

The project planned three major tasks which required restricted access to LLM text resources that needed supervision through administrative control together with User Interface improvements and the choice of implementing Kubernetes to reach the final production launch. Nirmal created an entire GPT-4o structured output engine that provided PII redaction and confidence scoring together with a reasoning audit chain which exceeded basic category suggestion capabilities. The project received additional features which included Google OAuth and GitHub Actions CI/CD pipeline to replace manual deployment errors which were found during the Sprint 2 retrospective.

Sprint 4 — Quality, Standards and Documentation (Weeks 7–8)

The project planned three major tasks which required restricted access to LLM text resources that needed supervision through administrative control together with User Interface improvements and the choice of implementing Kubernetes to reach the final production launch. Nirmal created an entire GPT-4o structured output engine that provided PII redaction and confidence scoring together with a reasoning audit chain which exceeded basic category suggestion capabilities. The project received additional features which included Google OAuth and GitHub Actions CI/CD pipeline to replace manual deployment errors which were found during the Sprint 2 retrospective.

6.4 Version History

Table 10. Version History

Version	Type	Sprint	Changes
v1.0.0	Major	Sprint 1	Initial release: auth, registration, ticket submission, Docker, AKS deploy
v1.1.0	Minor	Sprint 2	Admin dashboard, Google Maps, email notifications, Azure Blob Storage
v1.2.0	Minor	Sprint 3	GPT-4o AI integration, avatar upload, profile management, GitHub Actions CI/CD, Google OAuth
v2.0.0	Major	Sprint 4	QA verified (28/28 PASS), 3 Python decorators, Cultural Guardian, caching, audit logging, bilingual UI

7. Testing

7.1 Testing Strategy

The testing process involved both team members Rukshan De Silva who served as Frontend Lead and QA Nirmal Unagalle who served as Backend Lead conducted system tests during four testing sprints. The testing process implemented three testing levels which were executed in an organized manner.

The first testing method checks single components through unit tests which validate password strength regex and form field validation and decorator logic and ray-casting algorithm correctness. The second testing method verifies API endpoints through testing which includes authentication flows and database interactions and Azure Blob and email SMTP cross-service integrations. The entire end-to-end testing process for System testing includes the complete Sprint 1 to 4 QA checklist which contains 28 test cases that were executed on the live deployed Azure environment across all four sprints.

The test team conducted all 28 test cases through the Given/When/Then (GWT) specification format which follows Behavior Driven Development (BDD) rules while maintaining established pass/fail criteria which they documented before the testing began. The QA report presents complete testing results along with screenshots and a record of defects as separate supporting documentation.

7.2 Test Cases and Results

Table 11. Test Cases and Results

ID	Feature	Given / When / Then	Priority	Result
TC01	Registration	Given new user / When valid form submitted / Then account created, redirected to /login, welcome email sent	MUST	PASS
TC02	Password strength	Given registration / When password lacks uppercase or number / Then form blocked with descriptive error	MUST	PASS
TC03	Login — citizen	Given registered user / When correct credentials / Then JWT issued, redirect to /dashboard	MUST	PASS
TC04	Login — admin block	Given admin account / When citizen portal login attempted / Then gold banner shown, login blocked	MUST	PASS
TC05	Password toggle	Given login page / When eye icon clicked / Then password correctly shows/hides (DOM re-mount fix)	SHOULD	PASS
TC06	PrivateRoute guard	Given unauthenticated user / When navigates to /dashboard / Then immediately redirected to /login	MUST	PASS
TC07	Report submission	Given citizen logged in / When 3-step form completed / Then ticket created, confirmation email sent	MUST	PASS
TC08	GPS auto-fill	Given report form step 1 / When Use My Location clicked / Then lat/lng filled automatically	SHOULD	PASS
TC09	Photo upload	Given report step 2 / When photo selected / Then preview shown, upload accepted	SHOULD	PASS
TC10	AI analysis	Given ticket submitted / When GPT-4o completes / Then AILog row has crew, escalation, summary, confidence	SHOULD	PASS
TC11	Admin map	Given admin logged in / When map page opened / Then all tickets shown as coloured pins	SHOULD	PASS

TC12	Admin assign crew	Given admin on ticket / When crew assigned and saved / Then ticket updated in database	MUST	PASS
TC13	Avatar upload	Given citizen on dashboard / When avatar selected / Then preview shown, uploaded to Azure Blob	SHOULD	PASS
TC14	Cross-user avatar	Given user A with avatar / When user B logs in same browser / Then user B sees own avatar only	MUST	PASS
TC15	Avatar cache	Given user logged out / When new user logs in / Then browser does not serve old user avatar	MUST	PASS
TC16	Profile edit	Given citizen on dashboard / When name/email/phone updated / Then changes persisted to database	SHOULD	PASS
TC17	Password change	Given citizen on dashboard / When current + new password entered correctly / Then password updated	SHOULD	PASS
TC18	Email opt-out	Given user with email_notifications=false / When ticket status updated / Then no email sent	SHOULD	PASS
TC19	Email opt-in	Given user with email_notifications=true / When status updated / Then email sent with new status	SHOULD	PASS
TC20	Checkbox visibility	Given light mode browser / When registration page opened / Then email checkbox border visible	SHOULD	PASS
TC21	Admin stats	Given non-admin user / When /api/admin/stats/ accessed / Then 403 Forbidden returned	MUST	PASS
TC22	Public stats cache	Given public_stats called / When called again within 300s / Then X-Cache: HIT returned	COULD	PASS
TC23	Audit log	Given any request to map_tickets / When request completes / Then log entry written with user and timing	COULD	PASS
TC24	Cultural flag	Given report near Maungawhau polygon / When ticket submitted / Then cultural_flag=True on ticket	COULD	PASS
TC25	Google OAuth	Given Google account / When Sign in with Google clicked / Then citizen account created/matched and JWT returned	SHOULD	PASS

TC26	Rate limiting	Given anonymous user / When 11th ticket submitted within hour / Then 429 Too Many Requests returned	MUST	PASS
TC27	Token refresh	Given expired access token / When API called / Then 401 interceptor refreshes token automatically	MUST	PASS
TC28	Health check	Given /api/health/ called / When database and storage accessible / Then status: online returned	COULD	PASS

Figure 7.1 — QA Test Results Summary

7.3 Bug Fixes Identified During Testing

Table 12. Bug Fixes Identified During Testing

Bug	Root Cause	Fix	Commit
Password visible after toggle to hide	Browser reused DOM element for input, maintaining visible state	Added key prop to force full input DOM re-mount	ce046be
/dashboard accessible without auth	useEffect-based guard fires after render, brief unprotected access	Moved guard to PrivateRoute at router level	b5f8fd2
Cross-user avatar leak	localStorage C_PROFILE key persisted across user sessions	Added removeItem(C_PROFILE) in login() function	b5f8fd2
Checkbox invisible in light mode	Border colour used rgba(255,255,255,0.4) — white on white	Changed to CSS variable var(--border, #cbd5e1)	62b2124
Browser cached old user avatar	No cache-control headers on avatar proxy endpoint	Added Cache-Control: no-store to avatar response	d9cc26d
Eye icon inverted state	Eye/EyeOff icon logic was swapped in JSX condition	Corrected conditional rendering of icon components	ce046be

8. Technical Challenges and Innovative Solutions

8.1 GPT-4o AI Integration

Nirmal's GPT-4o integration performs more advanced functions than basic text processing. The AI system operates as a middleware system which maintains operational continuity because its background daemon thread conducts analysis after ticket storage to deliver 201 Created responses to citizens without any delay. The prompt uses temperature=0.2 to produce structured JSON output which maintains high consistency throughout its crew assignment and escalation level and plain-language summary and 0.0–1.0 confidence score components. The reasoning JSONField stores GPT-4o's decision-making process which enables full administrative audit capability of its entire decision-making path. The `redact_pii()` layer removes email addresses and phone numbers from ticket descriptions before they reach the OpenAI API which ensures NZ Privacy Act compliance when citizens accidentally provide their personal information.

8.2 Cultural Guardian Module

The Cultural Guardian uses a pure-Python ray-casting point-in-polygon algorithm to check every submitted GPS coordinate against Wāhi Tapu polygon boundaries because they need immediate results instead of waiting for post-hoc cultural sensitivity review. The four Auckland polygon stubs represent Maungawhau / Mount Eden, Orakei Basin, Mangere Mountain, and Otahuhu / Te Papapa. The admin interface shows an amber warning for flagged tickets which needs manual cultural review before any crew can be sent. The GeoJSON API from the active council system would provide the polygons used in production. The system implements Te Tiriti Protection as an architectural feature which operates during runtime instead of existing as a policy document.

8.3 Python Decorator Architecture

The three custom decorators handle cross-cutting concerns through declarative composition which avoids the need to duplicate code between different views. The `@log_request` decorator creates an audit record which includes both user identification and the duration of their response. The `@require_council_role` decorator blocks access to non-administrative users by returning a 403 status code. The `@cache_response` decorator with its timeout argument stores data in Django's cache system while the system sends an X-Cache header which indicates a cache hit or miss.

The parameterised `@cache_response` factory demonstrates advanced meta-programming because it produces a decorator which creates a wrapper function through three closure levels. The Open/Closed Principle supports this system because decorators enable views to obtain new functions without changing the view implementation. The public statistics endpoint achieves approximately 98% reduction in database load after `@cache_response(300)` was applied.

9. Software Evolution and Maintenance

9.1 Maintenance Activities

Table 13. Maintenance Activities

Type	Sprint	Activity	Commit
Corrective	Sprint 4	Fixed password toggle — key prop forces DOM re-mount	ce046be
Corrective	Sprint 4	Fixed cross-user avatar leak — clear C_PROFILE on login	b5f8fd2
Corrective	Sprint 4	Fixed /dashboard auth guard moved to PrivateRoute	b5f8fd2
Corrective	Sprint 4	Fixed checkbox border invisible in light mode	62b2124
Corrective	Sprint 4	Fixed avatar browser caching with Cache-Control: no-store header	d9cc26d
Adaptive	Sprint 3	Switched location search from Google Geocoder to Nominatim (free tier)	4b43fa5
Adaptive	Sprint 3	Added Azure Blob avatar proxy to avoid SAS URL expiry issues	f6d07b1
Perfective	Sprint 4	Added <code>@cache_response(300)</code> on <code>public_stats</code> endpoint	4f31472
Perfective	Sprint 4	Added <code>@log_request</code> audit logging on all key endpoints	4f31472
Preventive	Sprint 1	Rate throttling: 5/hr register, 10/hr ticket creation	views.py
Preventive	Sprint 2	PostgreSQL SSL enforcement (<code>sslmode=require</code>)	docker-compose

9.2 Version Control

The project uses Git as its development method to create separate branches for each developer while GitHub serves as their repository platform. My frontend and decorator work exists on the `rukshan` branch while Nirmal's backend and DevOps work resides on the `nirmal` branch. The development team prefers using the `--no-ff` option during main branch merges because it enables them to keep track of all branch activities while displaying all modifications made by different developers in the git log.

The commit messages use the conventional commits format which includes the prefixes fix and feat and refactor and test and they point to the particular issue or test case that they resolve.

9.3 Future Roadmap

- Spatial deduplication: cluster tickets within 50-metre radius to prevent duplicate reports for the same issue (geopy HaversineDistance)
- GPS EXIF extraction: automatically extract location from uploaded photo metadata using Pillow ExifData
- Expanded Wāhi Tapu database: integrate with live council GeoJSON API instead of hardcoded polygon stubs
- PII redaction engine: extend `redact_pii()` to cover NZ-specific phone number formats and street addresses
- Analytics dashboard: trend charts, heatmaps, and SLA reporting for council management using Chart.js
- Progressive Web App offline mode: queue reports in IndexedDB when offline and submit when connectivity restored

10. Limitations and Future Work

10.1 Known Limitations

- Wāhi Tapu polygons are representative stubs for four Auckland sites; production deployment requires live council GeoJSON data sourced from an authoritative Māori heritage register
- Avatar storage and AI analysis require active Azure and OpenAI credentials respectively; these features degrade gracefully if credentials are absent, but are not available in credential-free environments
- The application uses a single AKS replica per service; high-availability clustering would require multiple replicas and session affinity configuration for the Django cache layer

- Google Maps API costs scale with usage; heavy public map traffic at city-wide scale could incur significant API costs that would require a Maps for Work enterprise licence
- Email delivery relies on Namecheap SMTP credentials; production deployment should migrate to a dedicated transactional email service (SendGrid or AWS SES) for higher delivery reliability and volume

10.2 Future Enhancements

- Native iOS/Android applications using React Native sharing the same backend API, enabling push notifications and background location tracking
- Real-time ticket status updates using WebSockets (Django Channels) or Server-Sent Events instead of polling, reducing infrastructure load
- Machine learning demand forecasting to predict maintenance crew requirements by season and geographic area using historical ticket data
- Integration with council GIS systems for automated Wāhi Tapu polygon synchronisation and boundary updates without code changes
- Accessibility (WCAG 2.1 AA) audit and remediation pass to ensure the platform is usable by citizens with disabilities

11. Te Tiriti o Waitangi

11.1 Partnership — Kāwanatanga

The two team members shared equal power to make decisions for the project. The team consulted a Kaiārahi during requirements gathering to identify Māori community needs for reporting issues around culturally important sites. The document SPECIFICATIONS.md functioned as a common reference point which provided complete access to all design choices. The team selected Te Tiriti as a mandatory checkpoint for their sprint retrospectives because they required assessment of new features which would impact the three principles from their work with indigenous cultures.

11.2 Participation — Tino Rangatiratanga (My Contribution)

The React interface permanently shows Te Reo Māori because it serves as a default language option. The eight issue categories display bilingual labels which include 'Pothole / Roto Ara' as a complete submission method for anonymous users. The platform displays Māori language content permanently to show Māori users that the developers designed the system for their use from the initial development phase.

11.3 Protection — Ōritetanga

The Cultural Guardian module protects its system through executable code which tests every GPS coordinate against Wāhi Tapu polygon boundaries before dispatching crews.

I displayed the `cultural_flag` in the admin interface using an amber warning badge which required manual cultural review before any crew could be assigned. The system operates as a runtime guard which activates during ticket submissions to prove that software engineers must use cultural protection values as fundamental elements of their software design process.

12. Reflection

12.1 Technical Growth

The dual authentication context architecture represented the most difficult technical challenge for my professional work. The first implementation I created used `AuthContext` as a single authentication system which both citizen and admin portals shared and this system caused role-confusion problems. The system now functions with two separate context providers which maintain different `localStorage` key areas because I needed to create multiple storage areas to support different operational areas of the system. The systematic QA process taught me different methods of evaluating software because its 28 Given/When/Then cases required execution. I now use QA as a design process instead of a testing process.

My Python decorators implementation helped me understand Python object model behavior especially how closure scope works and how `functools.wraps` function operates. Nirmal's backend study provided me with practical experience about three new architectural patterns which include Recreate deployment strategy for avoiding simultaneous Django migration errors and PII redaction privacy engineering pattern and Kubernetes secret rotation process. I will become a better contributor to backend and DevOps work when I understand these decisions as software engineering choices instead of infrastructure choices.

12.2 Cultural Learnings

The project showed me that bilingual education supports language learning because students need to develop both their first and second languages. I previously viewed bilingual support as an internationalization i18n feature that needed development after completing the main product. The implementation of Te Reo Māori as a mandatory interface element required me to establish the user perspective that the product would prioritize from its initial development.

The Wāhi Tapu module demonstrated that engineers can create cultural protection through software engineering by developing privacy and accessibility and cultural safety features as core design elements. This principle will shape how I approach requirements in every future project.

12.3 Teamwork

Our team produced our most significant joint work through our work on SPECIFICATIONS.md which created vital agreements that established our data models and API contract shapes and token formats and error response structures. The missing contract first step caused teams to spend mid-sprint time to fix integration issues. The branch-per-developer PR review process established a continuous feedback system which benefited both partners because Nirmal provided better guidance on my error-handling with React performance patterns than I would have received from a single session. Code review serves as the main pathway through which team members share their hidden knowledge with each other.

13. Conclusion

The organization FixItPublic establishes its entire civic maintenance system through Azure Kubernetes Service which maintains continuous integration with GitHub Actions CI/CD while non-blocking GPT-4o AI system handles data processing to deliver intelligent ticket routing solutions for staff members who need to work on submitted tickets without interrupting citizen services. I created the entire React 18 user interface as Frontend Lead and QA Engineer which included two authentication methods and Google Maps functionality and three-step GPS and photo report submission and citizen dashboard with avatar management features and I performed QA testing to achieve 100% success on 28 BDD test cases after I found and fixed 6 confirmed defects. My three custom Python decorators show advanced skills in handling cross-cutting concerns because they adhere to the Open/Closed Principle.

The combination of Nirmal Unagalle's backend system with GPT-4o structured-output AI and the pure-Python Cultural Guardian and Google OAuth and AKS CI/CD automation capabilities created a platform that surpasses traditional student project requirements in both complexity and operational readiness.

The integration of Te Tiriti o Waitangi principles as genuine runtime architectural features — not compliance checkboxes represents a standard of bicultural software engineering I will carry into professional practice.

14. References

References are formatted in accordance with APA 8th edition.

Christie, T. (2023). Django REST framework (Version 3.14). Encode OSS Ltd.

<https://www.django-rest-framework.org/>

Holovaty, A., & Willison, S. (2005). Django: The web framework for perfectionists with deadlines (Version 4.2). Django Software Foundation. <https://www.djangoproject.com/>

Jazzband. (2023). djangorestframework-simplejwt: A JSON Web Token authentication plugin for Django REST Framework (Version 5.3). <https://django-rest-framework-simplejwt.readthedocs.io/>

Meta Open Source. (2023). React: A JavaScript library for building user interfaces (Version 18). Meta Platforms, Inc. <https://react.dev/>

Microsoft Corporation. (2024). Azure Kubernetes Service (AKS) documentation. <https://learn.microsoft.com/en-us/azure/aks/>

Microsoft Corporation. (2024). Azure Container Registry documentation. <https://learn.microsoft.com/en-us/azure/container-registry/>

Microsoft Corporation. (2024). Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>

OpenAI. (2024). GPT-4o model card and system card. <https://platform.openai.com/docs/models/gpt-4o>

Google LLC. (2024). Maps JavaScript API overview. Google Developers. <https://developers.google.com/maps/documentation/javascript>

Google LLC. (2024). google-auth: Google authentication library for Python (Version 2.x). <https://google-auth.readthedocs.io/>

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), 70–93. <https://dl.acm.org/doi/10.1145/2898442.2898444>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- GitHub, Inc. (2024). *GitHub Actions documentation: Automate your workflow*. <https://docs.github.com/en/actions>
- New Zealand Government — Ministry of Health. (2019). *Te Tiriti o Waitangi principles in policy, planning, and practice*. <https://www.health.govt.nz/our-work/populations/maori-health/te-tiriti-o-waitangi>
- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8: Style guide for Python code*. Python Software Foundation. <https://peps.python.org/pep-0008/>
- Tailwind Labs Inc. (2024). *Tailwind CSS: A utility-first CSS framework (Version 3)*. <https://tailwindcss.com/docs>
- Lutz, M. (2013). *Learning Python (5th ed.)*. O'Reilly Media.
- Unagalle, N., & De Silva, R. (2026). *FixItPublic — Civic issue reporting platform [Source code]*. GitHub. <https://github.com/Nirmal-C/FixItPublic>
- De Silva, R. (2026). *MSE800 Python decorator assessment report*. GitHub. <https://github.com/Nirmal-C/FixItPublic/blob/main/docs/decorator-report.md>

15. Appendix

15.1 GitHub Repository

Main repository: <https://github.com/Nirmal-C/FixItPublic>

Decorator report: <https://github.com/Nirmal-C/FixItPublic/blob/main/docs/decorator-report.md>

CI/CD pipeline: <https://github.com/Nirmal-C/FixItPublic/blob/main/.github/workflows/deploy.yml>

15.2 Live Deployment

Production URL: <https://fixitpublic.com/>

Admin portal: <https://fixitpublic.com/admin>

15.3 Key Source Files

backend/api/decorators.py — @log_request, @require_council_role, @cache_response

backend/api/signals.py — post_save signal, GPT-4o daemon thread AI triage

backend/api/cultural_guardian.py — ray-casting point-in-polygon, 4 Wāhi Tapu site polygons

backend/api/google_auth.py — Google OAuth ID token verification and citizen account creation

frontend/src/contexts/CitizenAuthContext.jsx — Citizen JWT state management, Google OAuth, profile caching

frontend/src/utils/authUtils.js — Shared: decodePayload, isTokenExpired, clearAuthTokens

.github/workflows/deploy.yml — GitHub Actions CI/CD: build-and-push + deploy jobs

15.4 Git Contribution Log (Selected Key Commits)

The following is a representative selection of Git commits from the FixItPublic repository (github.com/Nirmal-C/FixItPublic) confirming both team members' contributions across all four sprints. Rukshan De Silva's commits cover the React frontend, QA fixes, Python decorators, and Cultural Guardian implementation.

Nirmal Unagalle's commits cover the Django backend, CI/CD pipeline, GPT-4o AI integration, Azure infrastructure, and DevOps automation.

Nirmal-C (Feb 15 – Mar 9, 2026): Initial commit; CI/CD workflow for Docker build and AKS deploy; Azure Blob Storage; JWT-based auth; health check endpoints; Nginx reverse proxy; Kubernetes manifests; ticketing system integration.

Rukshan De Silva (Mar 13 – 14, 2026): Sprint 1 frontend; Sprint 2 admin dashboard + GPS, map, chart, CSV, timeline, tracker; Te Reo Māori integration; light mode default; UI redesign.

security hardening (CORS, headers, secret key, password complexity, raw SQL fix); fix AI log pagination to return all entries.



15.5 Monday.com Sprint Board

Sprint backlog, task allocation, story points, and burndown charts were maintained on Monday.com throughout all four sprints.

15.6 QA Report

A separate QA Test Report document containing the full test case evidence, screenshots, defect log, and sign-off is submitted as a companion file to this report.